

Desarrollo Web para portal de Inteligencia Analítica

Lección 5. Conexión a base de datos no relacionales y consumo de base de datos relacionales

Introducción

¿Sabías qué las aplicaciones web modernas consumen grandes volúmenes de datos de manera interactiva? Y no solo desde archivos **CSV**, muchas también se conectan a bases de datos comerciales para leer, insertar, actualizar o eliminar información en tiempo real.

Afortunadamente, en **Python** podemos usar **Streamlit**, un *framework* que facilita la creación de aplicaciones web interactivas, así como la conexión a diferentes fuentes de datos para realizar lecturas, inserciones, actualizaciones o eliminación de datos en la nube de forma eficiente.



Por eso, estudiaremos los conceptos básicos de bases de datos, cómo diseñar una base en **Firestore**, y cómo migrar tus fuentes de datos desde archivos **CSV** a la nube, asegurando una estructura más robusta, segura y escalable.

Con estos conocimientos, podrás transformar tus aplicaciones en soluciones conectadas y dinámicas.

Consumo de base de datos no relacionales Firestore y MongoDB

Streamlit y **Python** permiten la conectividad a la base de datos **Firestore** de manera muy sencilla y con pocas líneas de código. Como científico de datos, es importante que aprendas a modelar bases de datos en **Firestore**, y a utilizar **Streamlit** para migrar datos **CSV** a la base de datos modelada en **Firestore**.

Firestore de Google es una plataforma en la nube para el desarrollo de aplicaciones web y móviles, actualmente se encuentra disponible para plataformas como: **iOS**, **Android**, **Web** y **Unity**. Fue creada en 2011 y en 2014 pasó a ser parte de **Google**, comenzando como una base de datos en tiempo real. Sin embargo, se añadieron más y más funciones, que, en parte, permitieron agrupar los **SDK** de productos de **Google** con distintos fines, facilitando su uso.

La función principal de **Firestore** es simplificar la creación y desarrollo de aplicaciones web, procurando que el trabajo sea más rápido, pero sin renunciar a la calidad requerida. Esto es especialmente interesante para que los desarrolladores no dediquen tanto tiempo al *backend*, tanto en cuestiones de desarrollo como de mantenimiento. **Firestore** se compone por una *suite* de productos que se muestran a continuación:



Suite de productos	Especificaciones
1. <i>Real Time</i>	Firestore Realtime es una base de datos NoSQL alojada en la nube que te permite almacenar y sincronizar datos en tiempo real. La sincronización en tiempo real permite que los usuarios accedan a sus datos desde cualquier dispositivo, web o móvil, con facilidad, y los ayuda a trabajar en conjunto. Si un usuario pierde conectividad de internet, los SDK de Realtime Database usan la caché local del dispositivo para publicar y almacenar cambios y cuando el dispositivo se conecta, los datos locales se sincronizan de manera automática.
2. Cloud Firestore	Cloud Firestore te permite almacenar, sincronizar y realizar búsquedas inteligentes de datos en una arquitectura altamente escalable.
3. Firestore ML	Firestore ML es un producto que incorpora un conjunto de APIs de inteligencia artificial para los casos de uso más comunes como: reconocer texto, etiquetar imágenes y reconocer puntos de referencia. A diferencia de otras API integradas en el dispositivo, estas aprovechan la potencia de la tecnología de aprendizaje automático de Google Cloud para brindarte un nivel alto de precisión. Simplemente se transfieren los datos a la API y se creará una solicitud sin interrupciones para los modelos que se ejecutan en Google Cloud y recibirás la información que necesitas con pocas líneas de código.

4. Cloud functions

Cloud Functions permite crear funciones de *backend* en **JavaScript** para resolver problemas específicos, las cuales se ejecutan en un entorno de **Node.js** seguro y administrado y solo se ejecutan cuando se emite un evento específico bajo observación. **Cloud functions** se activan con productos de **Firebase**, como cambios en los datos de **Realtime Database**, el registro de usuarios nuevos mediante **Auth** o los eventos de conversión en **Analytics**.

5. Hosting

Con **Firebase Hosting**, puedes publicar un sitio web completo sin complicaciones solo contando con la versión de producción en **HTML, CSS y Javascript**, sin importar el *framework* con el que fueron creados como **Jquery, Angular, React** o **Vue**. El contenido se publica rápidamente, sin importar la ubicación del usuario, los archivos implementados en **Firebase Hosting** se almacenan en servidores en la nube de **Google**.

6. Cloud Storage

Cloud Storage es un servicio de almacenamiento de objetos como imágenes, videos, audios, PDF, XML, entre otros. Es potente, simple y rentable, este servicio agrega seguridad a las operaciones de carga y descarga de archivos de las aplicaciones de **Firebase**, sin importar la latencia de la red. **Cloud Firestore** es la base de datos adecuada para el proyecto, ya que es una base de datos flexible y escalable para el desarrollo en servidores, dispositivos móviles y web desde **Firebase y Google Cloud**.

Más adelante, revisarás cómo crear una base de datos simple en **Firestore** que permita leer, escribir, borrar y actualizar datos. Continúa para conocer cómo hacerlo.

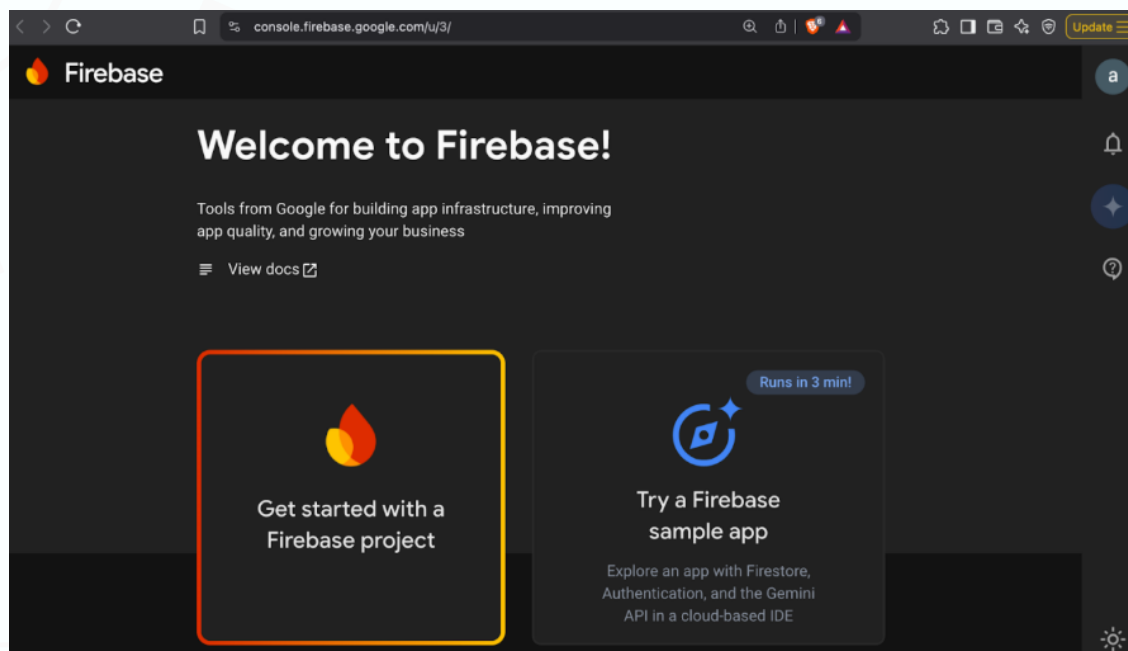
Entrenamiento para la configuración de Firebase

Es momento de poner manos a la obra en la **creación de una base de datos simple en Firestore**. Para lograrlo, es necesario realizar lo siguiente:

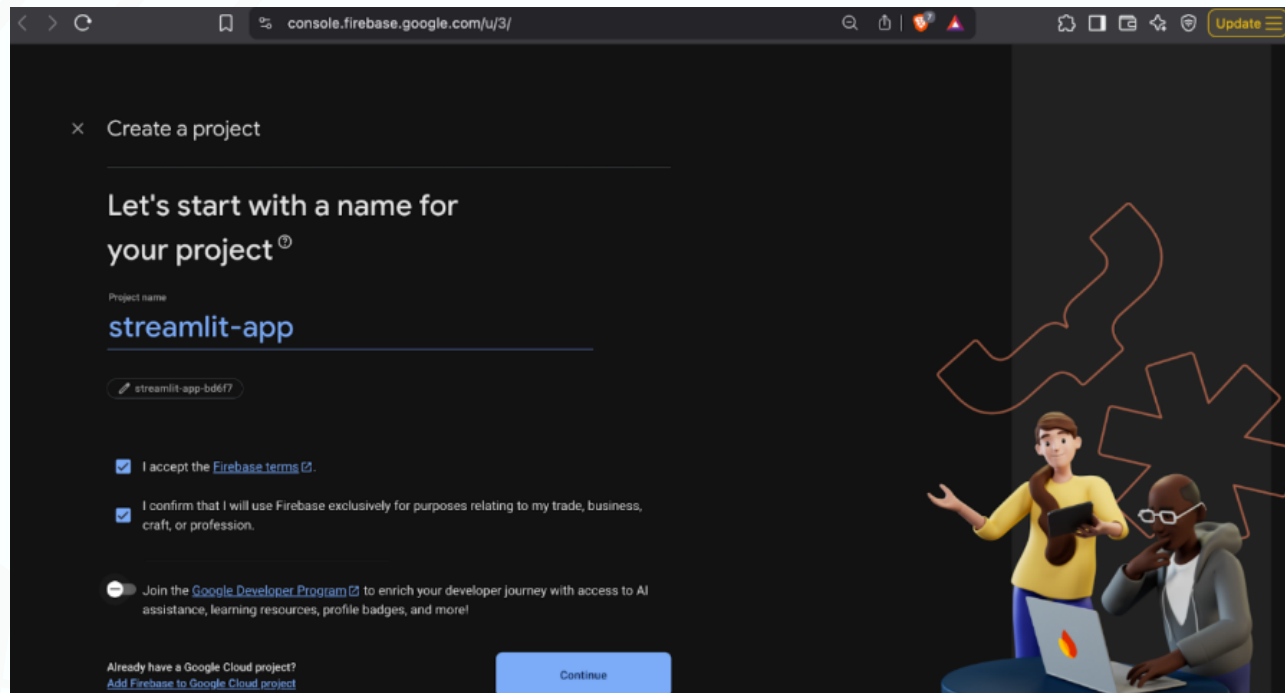
Paso 1. Dirígete a la consola de administración de **Firebase**. Para hacerlo, copia y pega el enlace en tu navegador: <https://console.firebase.google.com/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 12 de junio de 2025 y su disponibilidad depende del autor.

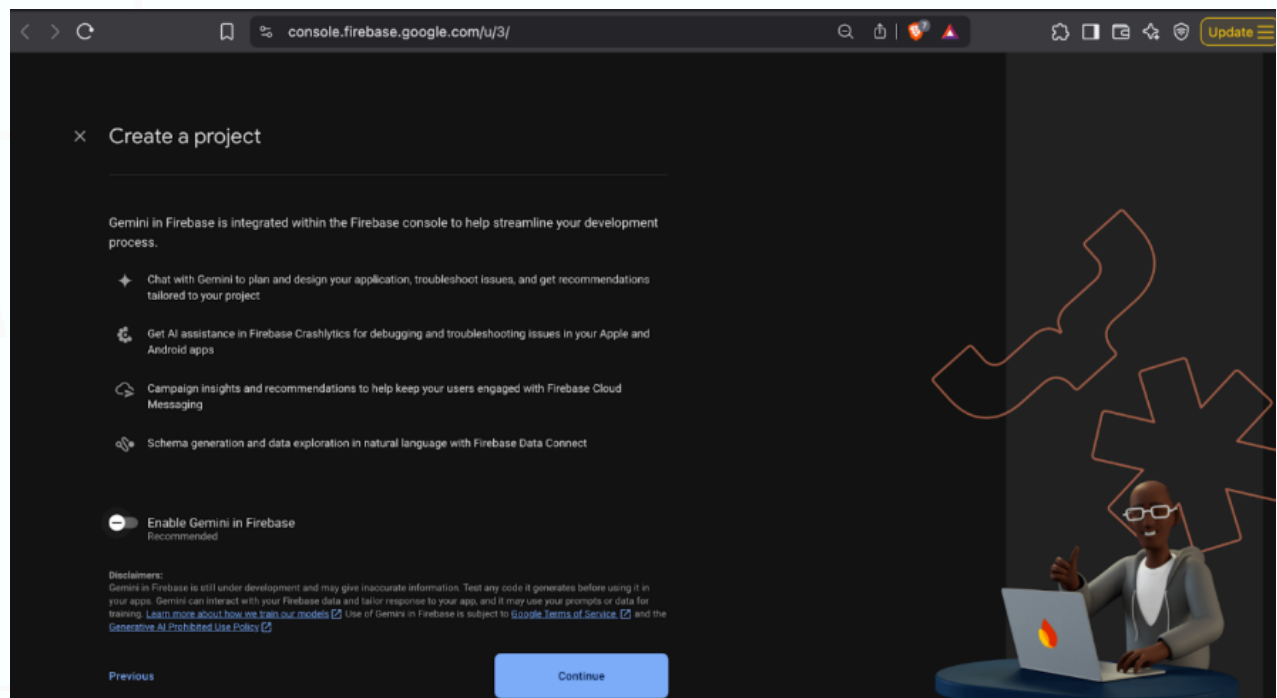
Importante: Si ya tienes cuenta de Gmail o institucional, Firebase la tomará por *default*.



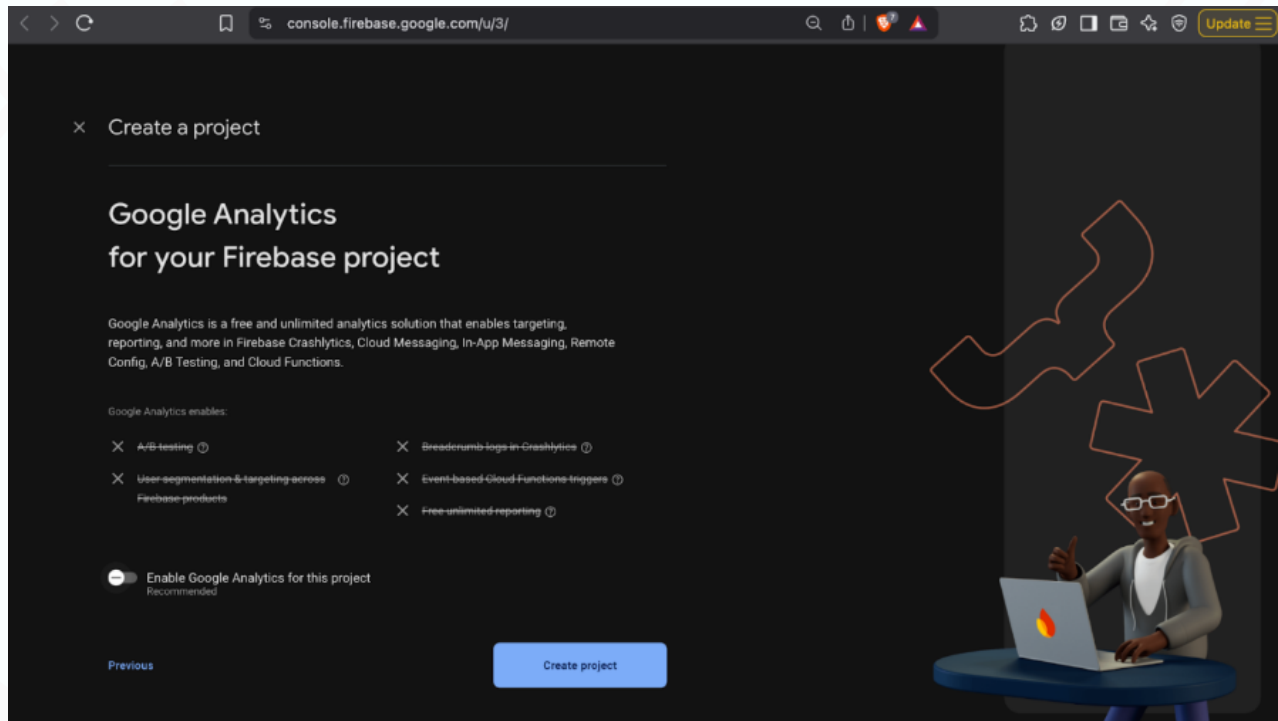
Paso 2. Elige la opción de **iniciar un proyecto nuevo**, y nómbralo **streamlit-app**.



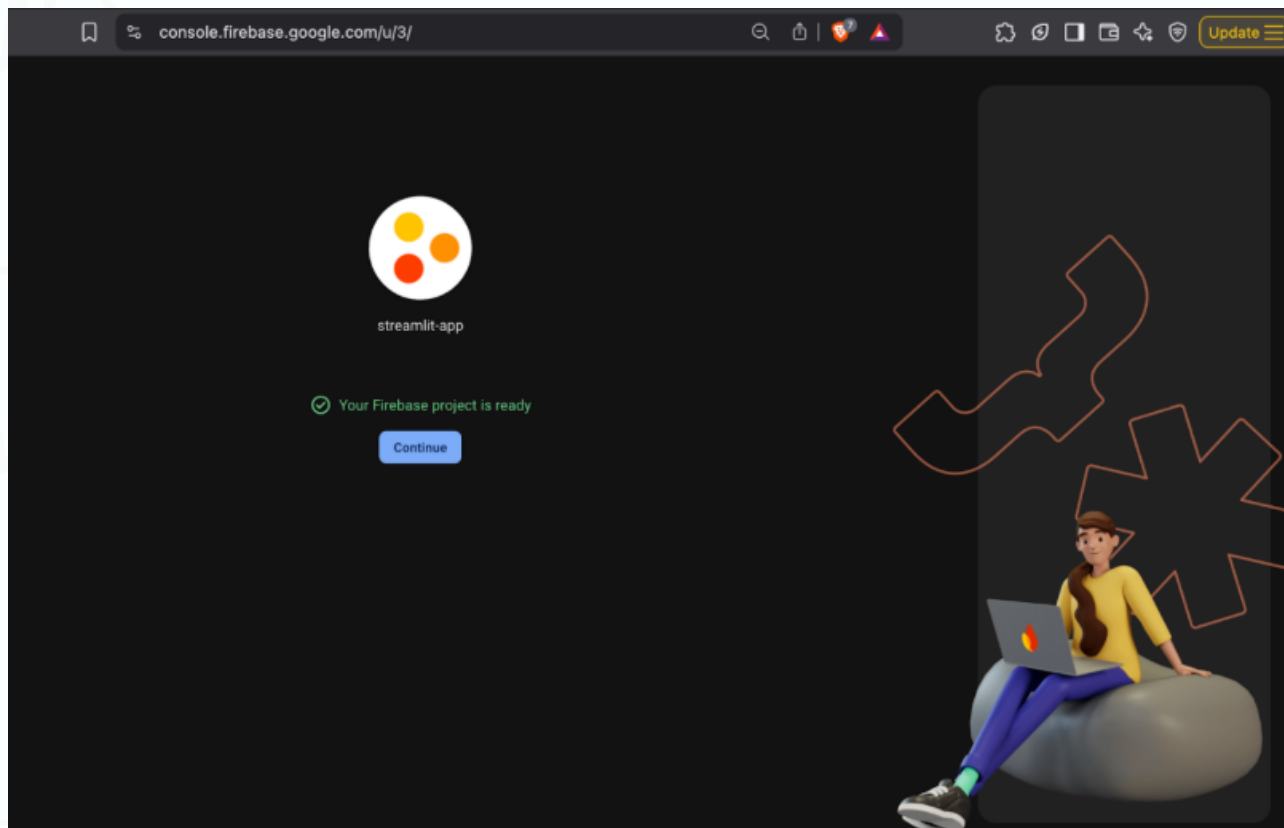
Paso 3. Deshabilita la opción de **Gemini in Firebase**.



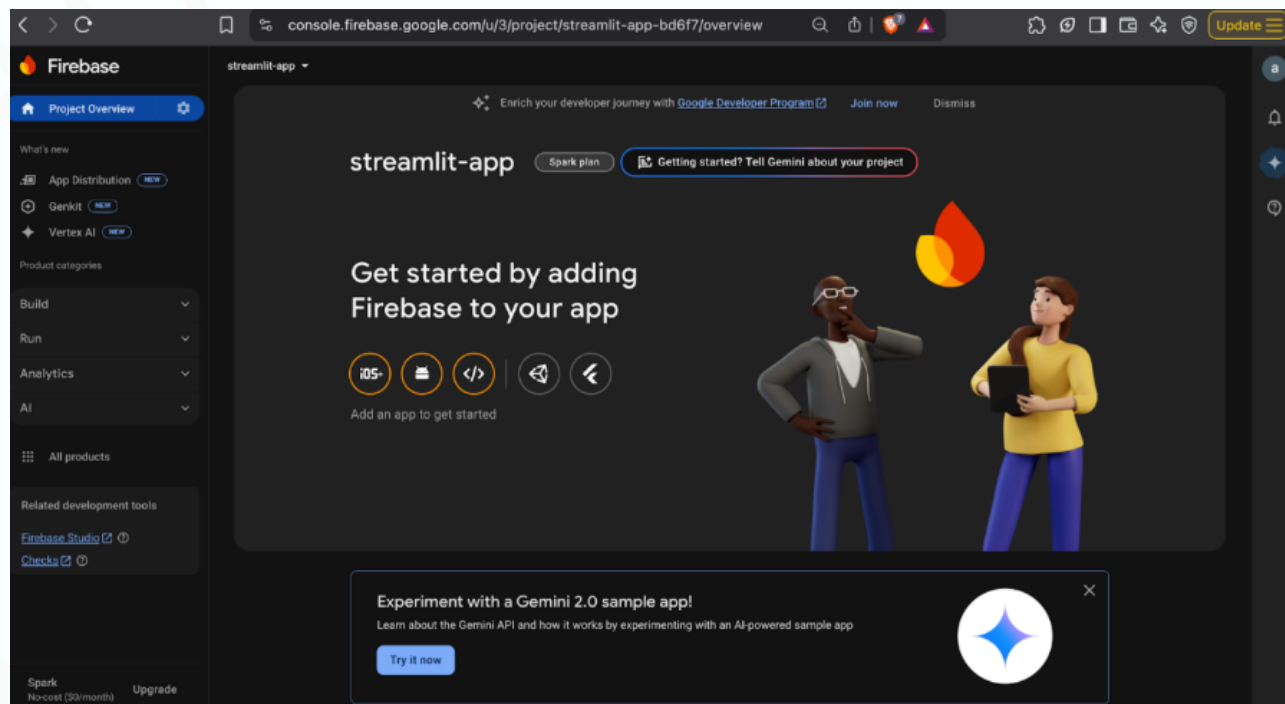
Paso 4. Análogamente, deshabilita **Google analytics** en el proyecto.



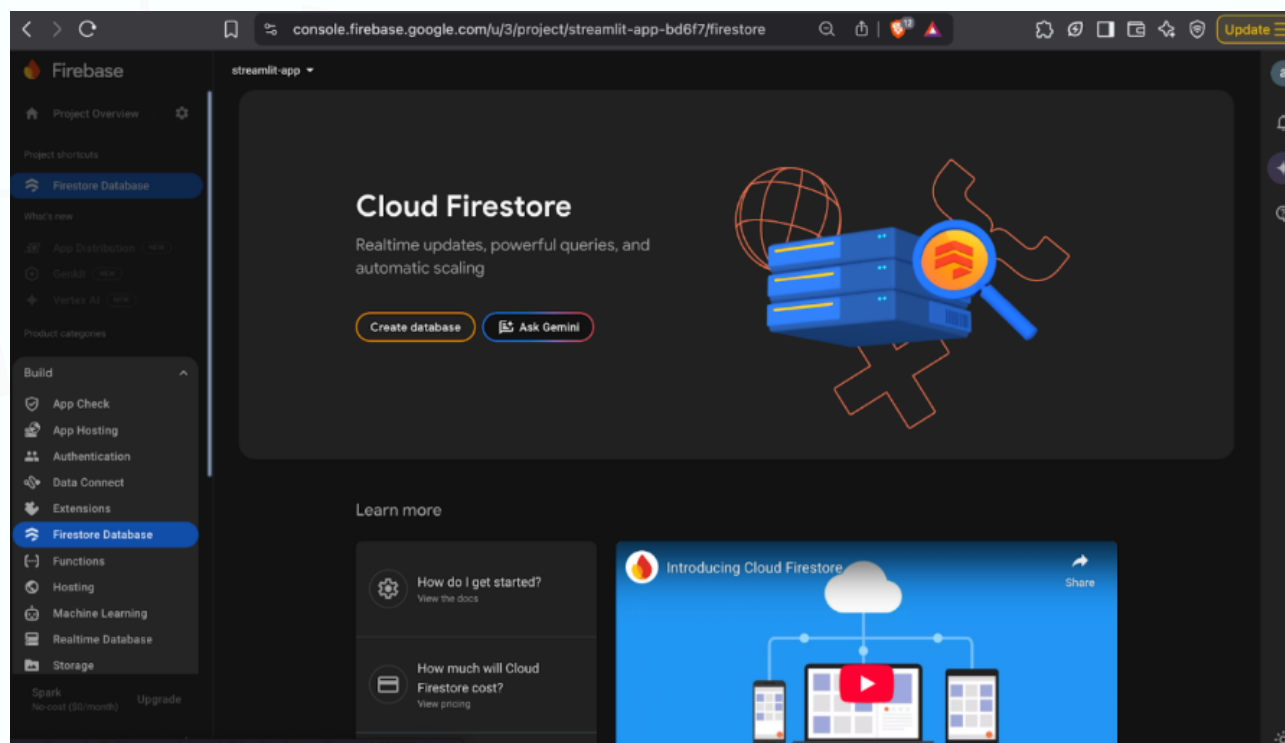
Paso 5. Ahora, observa que el proyecto estará listo para ser usado.



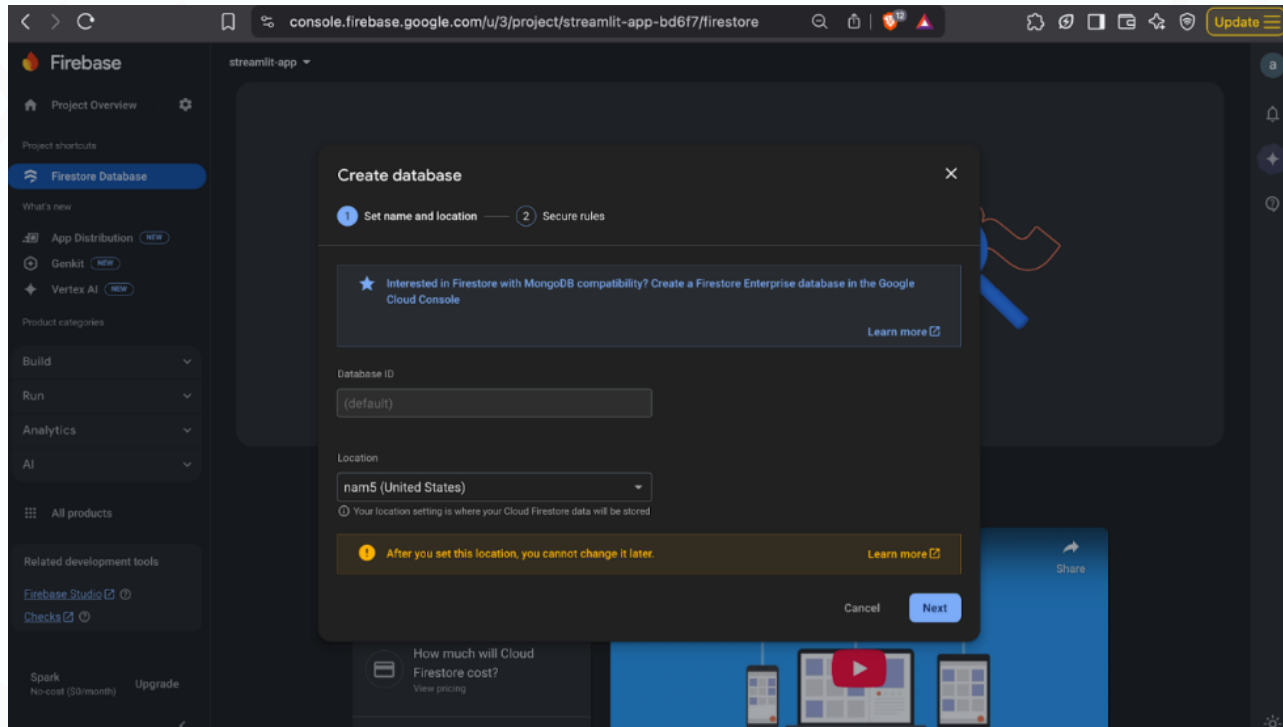
Paso 6. Comprueba la pantalla principal del proyecto, misma que lucirá así:



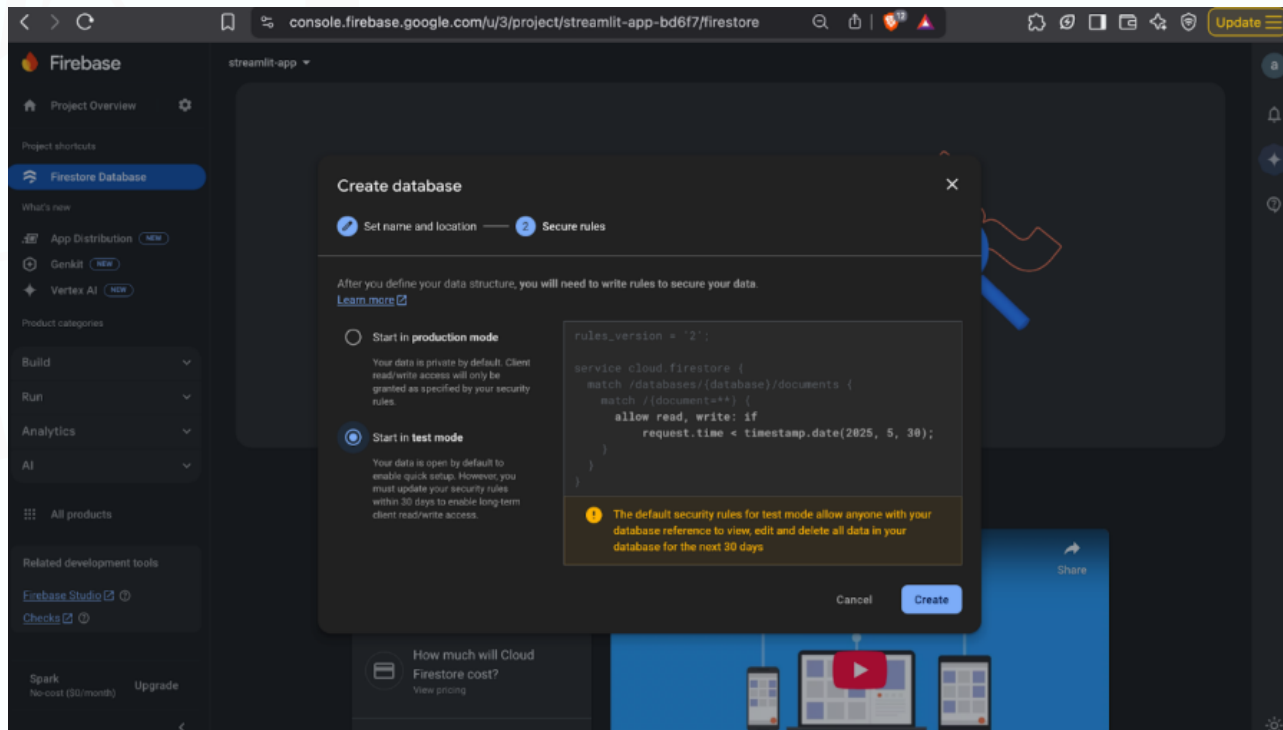
Paso 7. Posteriormente, selecciona la opción **Firestore** dentro del menú **Build** y haz clic en **Create database**.



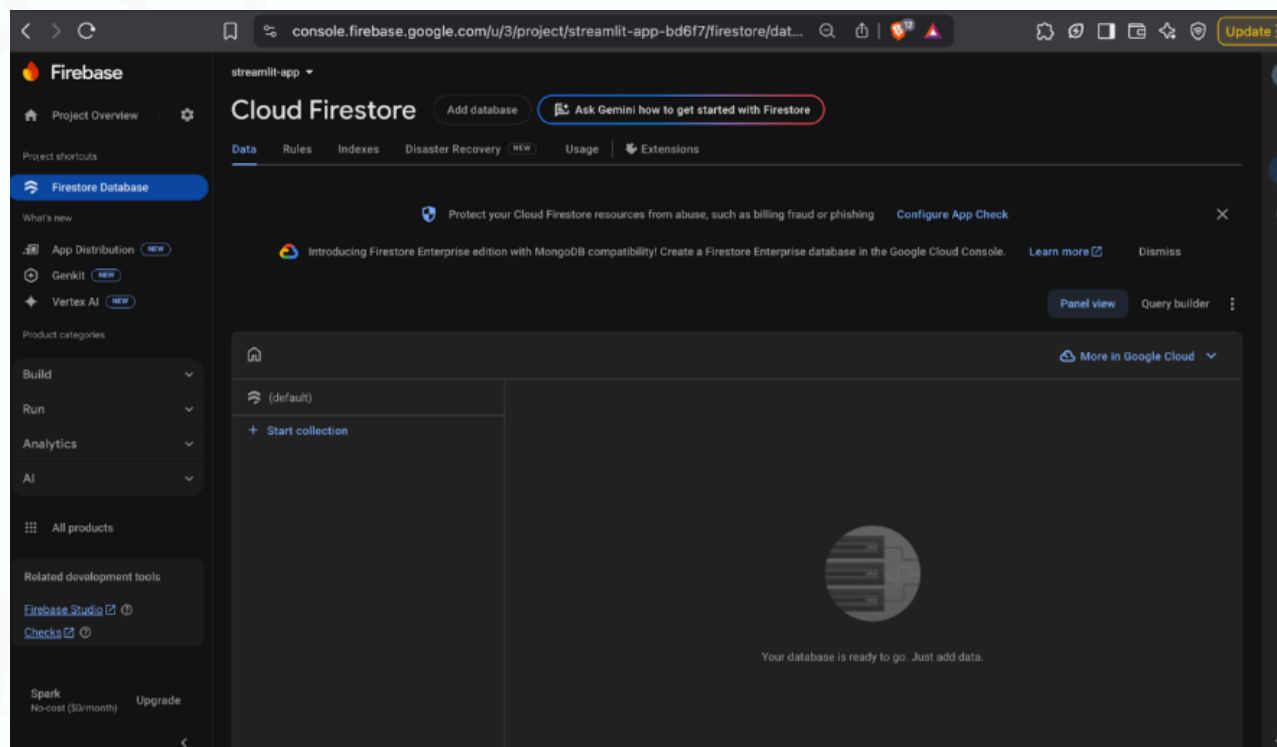
Paso 8. Crea una base de datos en la opción **Firestore Database**.



Paso 9. Elige el modo iniciar en modo **test** la base de datos.



Paso 10. Una vez creada la base de datos, estás listo para crear **colecciones**.



Cloud Firestore es una base de datos **NoSQL** orientada a los documentos. A diferencia de una base de datos **SQL**, no hay tablas ni filas, en su lugar, **almacena los datos en documentos que se organizan en colecciones**.

En ese sentido, cada **documento** contiene un conjunto de pares clave-valor. **Cloud Firestore** está optimizado para almacenar grandes colecciones de documentos pequeños, a su vez, todos los documentos se deben almacenar en colecciones, y pueden contener subcolecciones y objetos anidados. Además, ambos pueden incluir campos primitivos, como *strings*, o tipos de objetos complejos, como listas.

Documentos

En **Cloud Firestore**, la unidad de almacenamiento es el **documento**: un registro que usa pocos recursos y contiene campos con valores asignados. **Cada documento se identifica con un nombre específico o ID generado automáticamente.** Un documento que representa a un registro de *names* puede tener el siguiente aspecto:

```
index: 314  
  
name: "Priscilla"  
  
sex: "F"
```

Además, **Cloud Firestore** es compatible con diversos tipos de datos para los valores, como **booleanos, números, strings, puntos geográficos, BLOB binarios y TimeStamp**. Adicionalmente, podemos usar **arreglos u objetos anidados**, llamados **mapas**, para estructurar datos dentro de un documento. Por ejemplo, podrías estructurar el nombre del usuario del ejemplo anterior con un mapa como el siguiente:

```
index: 314  
  
▼ name  
    firstname: "Priscilla"  
    lastname: "Smith"  
  
sex: "F"
```

Cabe mencionar que un documento en Firestore es muy similar a un JSON; básicamente son **JSON**. Sin embargo, existen algunas ligeras diferencias, por ejemplo, que los documentos en **Firestore** admiten tipos de datos adicionales y su tamaño se limita a 1 MB, pero regularmente se pueden considerar los documentos como registros **JSON**.

Creación de colecciones

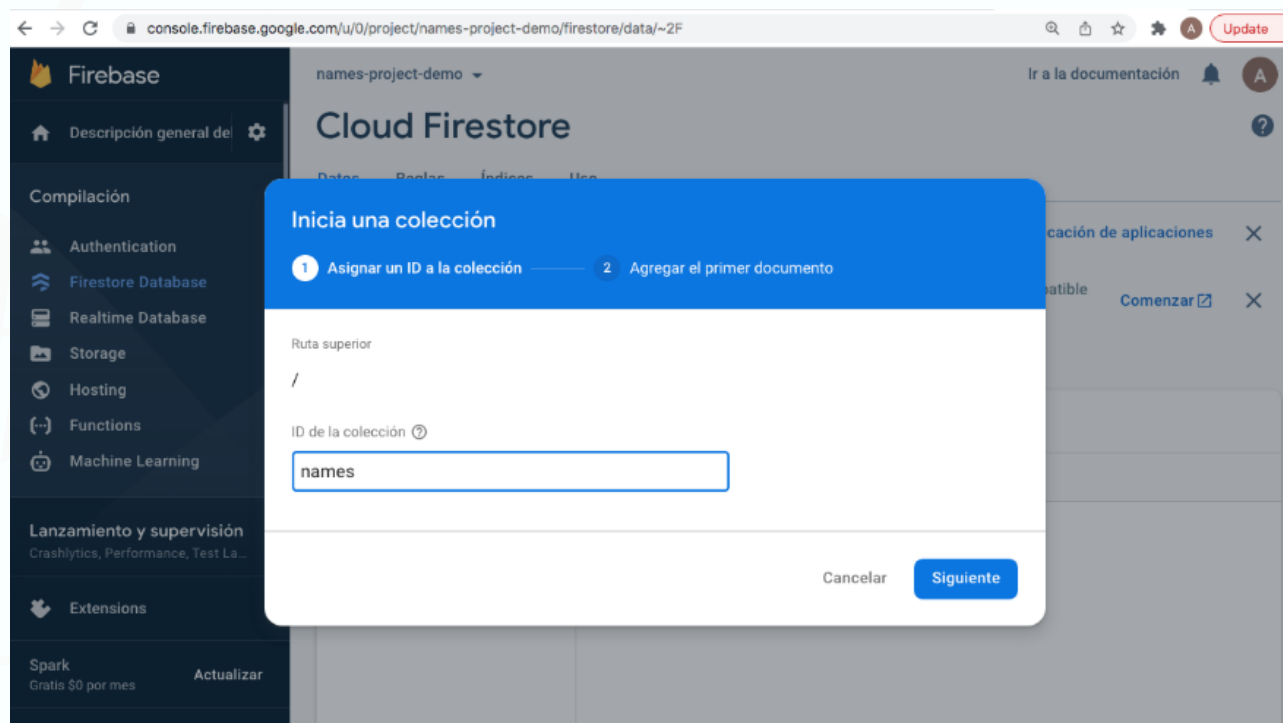
Los documentos se agrupan en **colecciones**, que simplemente son contenedores de estos. Por ejemplo, podrías tener una colección llamada **names** con diversos registros de **nombres** con un **ID** único que contenga un documento que represente a cada uno de la siguiente manera:

+ Iniciar colección	+ Agregar documento	+ Iniciar colección
names >	07ftJR9xwUUCHyjlxIvE	+ Agregar campo
	09YEpsre6Ka3Pzr6VeYW	index: 74
	0DUm7LTx8CeLkUjMCHuB >	name: "Caroline"
	0M7kuYnEMmQFuSQ8Ippu	sex: "F"
	0MAfiMa1410QzFgo3EXI	
	0W9apk3G7Un7r94nb8Lm	
	15E5JYwqDdMImx3oR0yn	
	16LnmXoe4sDyyMwthRZN	
	1I9n2XX7A1Kb5PLJ0mEJ	
	1Mg2A99Ta6CjqmuKK6L4	
	1hLiQVGrS8FKGdX7vcqd	
	1nM7DC3jM5UcTWadDEew	
	1pHessPP5MB52C2KhoOp	

En ese escenario, con **Cloud Firestore** tienes libertad total sobre los campos que asignes en cada documento y los tipos de datos que almacenes en esos campos. Los documentos dentro de una misma colección pueden contener campos diferentes o almacenar distintos tipos de datos en esos campos. Sin embargo, se recomienda usar los mismos campos y tipos de datos en varios documentos, de manera que puedas consultarlos con mayor facilidad. Por otra parte, los nombres de documentos dentro de una colección son únicos; puedes proporcionar tus propias claves, como los **ID** de usuario, o dejar que **Cloud Firestore** genere **ID aleatorios** automáticamente.

Es momento de crear una **Colección de Firebase** para almacenar documentos de nombres de acuerdo con la estructura **names.csv**, mencionada previamente, y que puedes descargar en el apartado archivos adjuntos de la plataforma.

1. **index**
2. **name**
3. **sex**



Para hacerlo, sigue estos pasos:

Paso 1. Crea un documento con la estructura de **names**, para esto, da clic en generar **ID automático** e ingresa los tres campos de la estructura **names**:

1. **index** tipo **number**
2. **name** tipo **string**
3. **sex** tipo **string**

Ruta superior del documento ?

/names

ID de documento ?

34v9P6Ox9SZ7jzIwrRFP

Campo	Tipo	Valor
index	number	0
name	string	Juan
sex	string	M

+ Agregar campo

Cancelar Guardar

Paso 2. Al hacerlo, observa que la colección **names** con el nuevo documento lucirá de la siguiente manera:

names-project-demo Cloud Firestore		
names 34v9P6Ox9SZ7jzIwrRFP		
+ Iniciar colección	+ Agregar documento 34v9P6Ox9SZ7jzIwrRFP >	+ Iniciar colección + Agregar campo index: 0 name: "Juan" sex: "M"

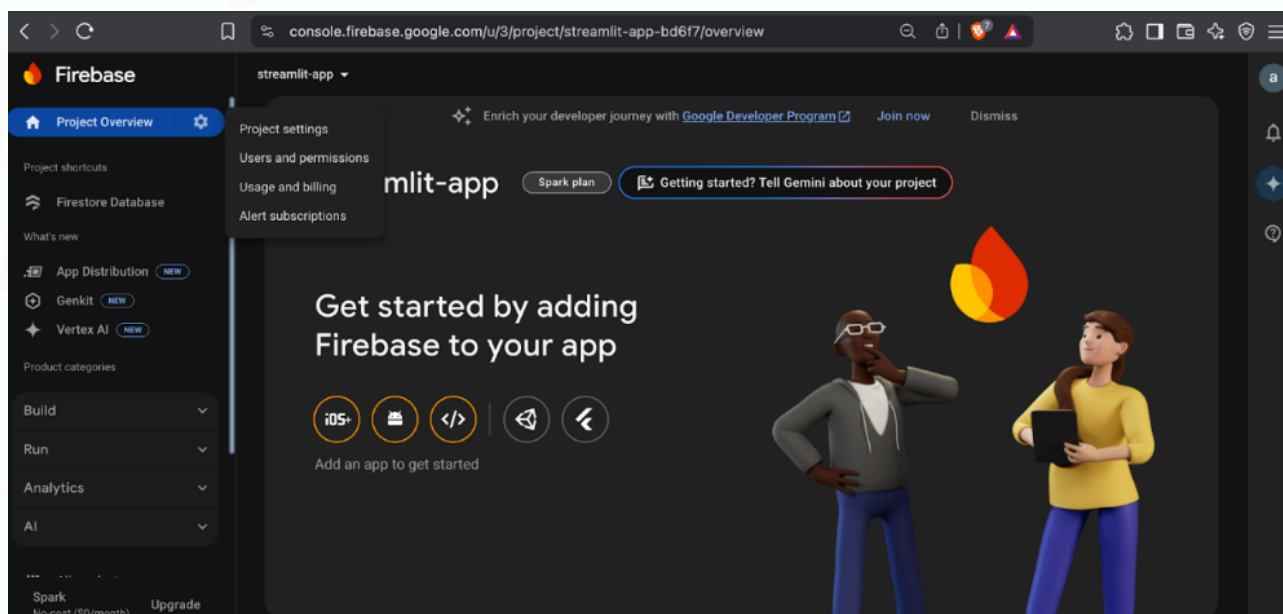
Error al cargar las colecciones

Paso 3. Una vez creada la base de datos, lo siguiente es descargar las **keys**. Continúa para descubrir cómo hacerlo.

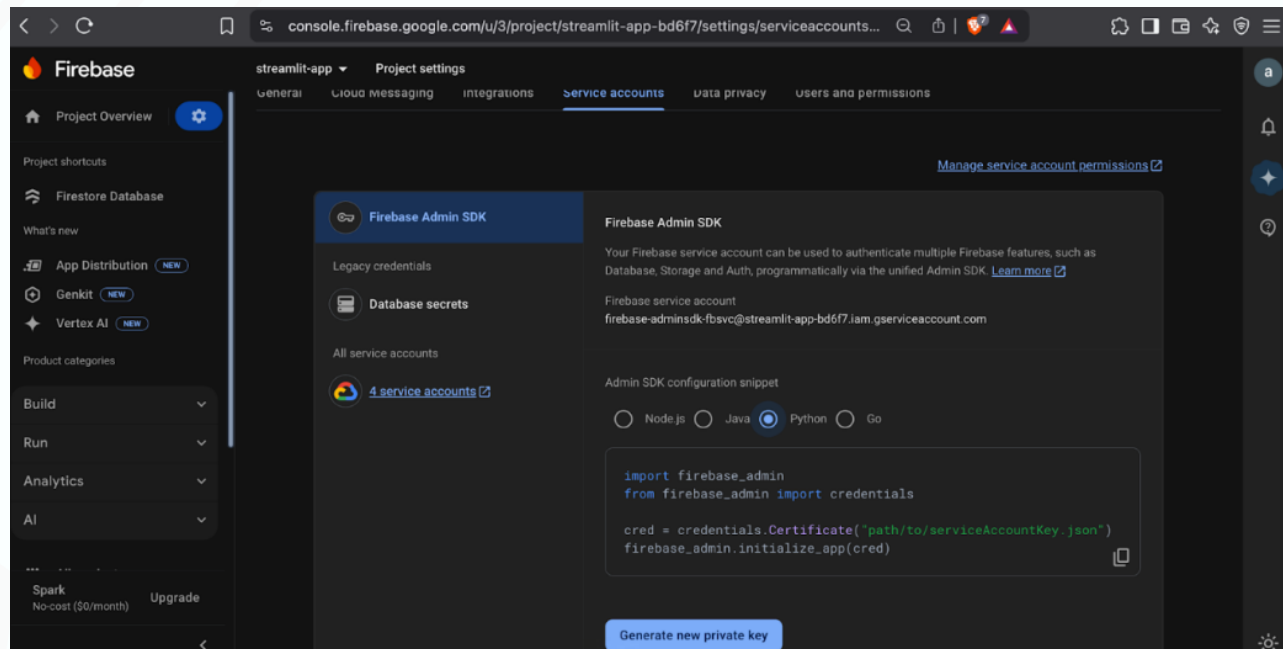
Descargar keys

Con la base de datos y la colección creada, el siguiente paso consiste en descargar las credenciales del proyecto de **Firebase**, mismas que usarás para conectar la aplicación en **Python** a la base de datos **Firebase**. Para hacerlo, continúa con los pasos:

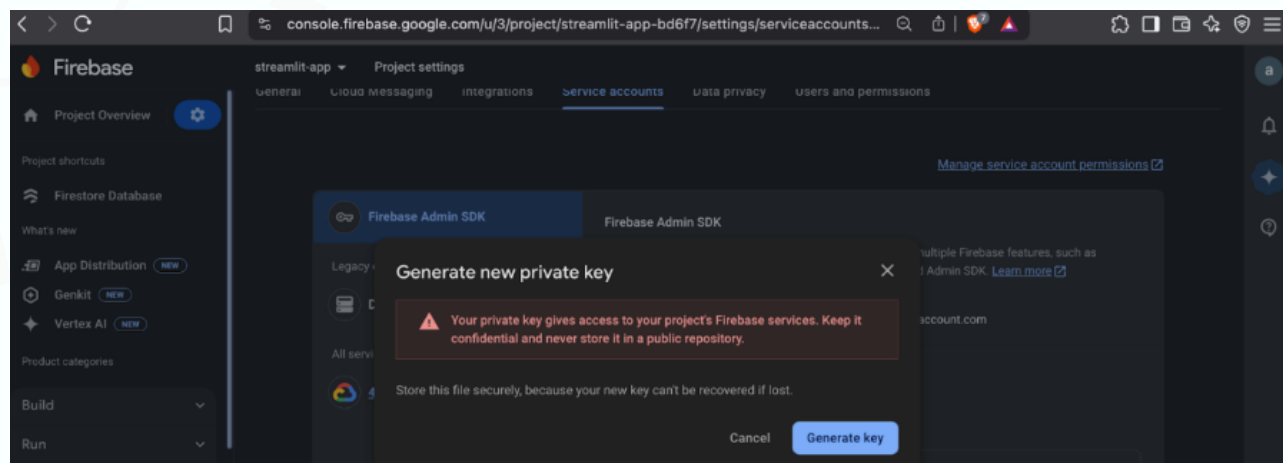
Paso 4. Da clic en **Descripción general del proyecto > Configuración del proyecto**



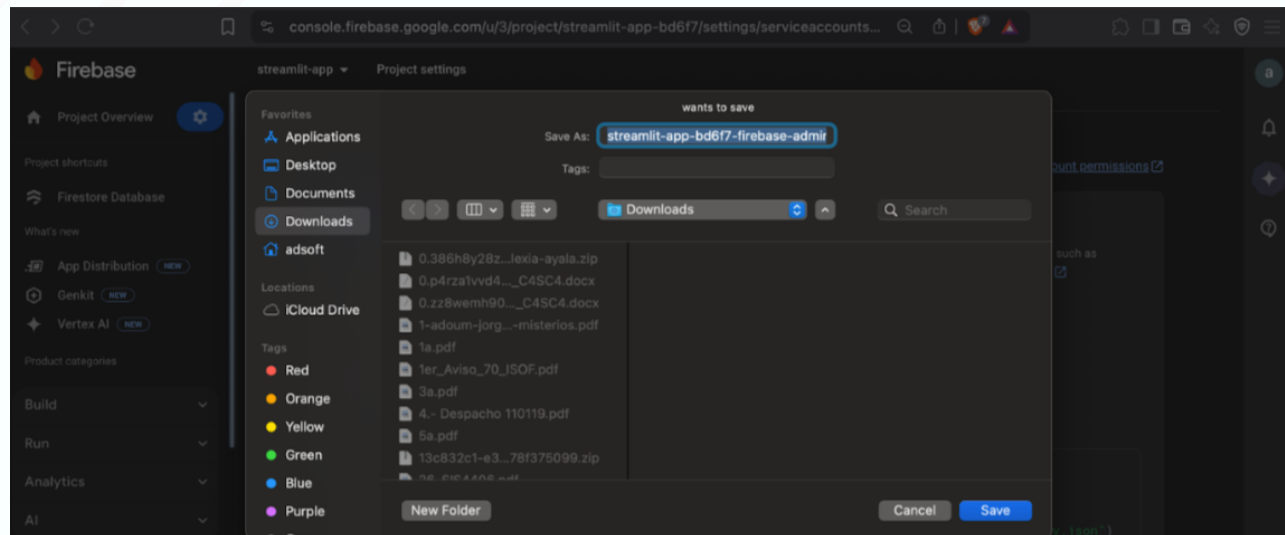
Paso 5. Selecciona **Cuentas de servicio**, Lenguaje de programación **Python** y Generar nueva clave privada.



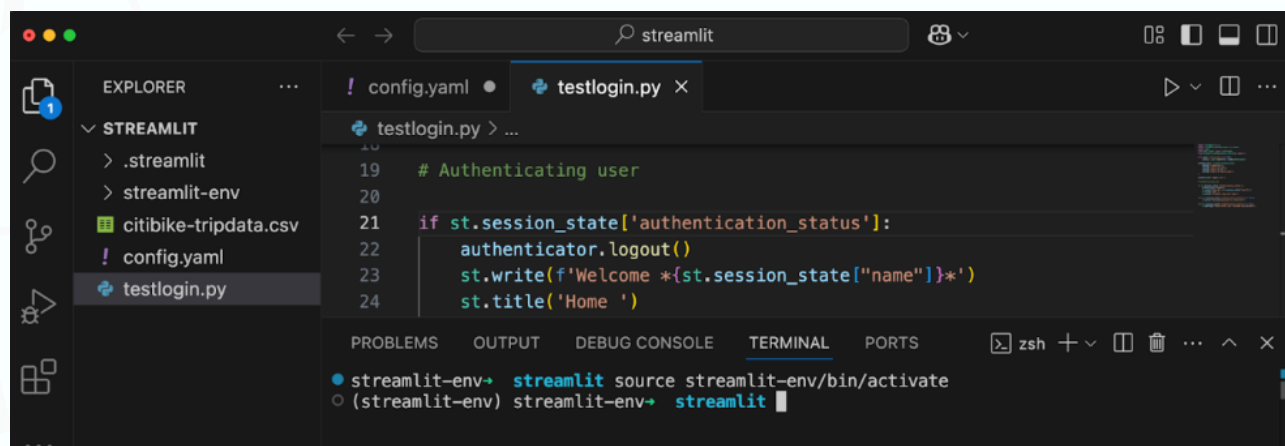
Paso 6. Observa que, al hacerlo, aparecerá un cuadro de diálogo, indicando que la clave privada es el acceso a los servicios de **Firebase**, misma que debes guardar con precaución, no compartirlas con nadie y nunca guardarlos en un repositorio de **git** público.



Paso 7. Este proceso descargará un archivo que incluye las credenciales con el formato **nombre-tu-proyecto.json**, posteriormente, lo usarás como llave de acceso.

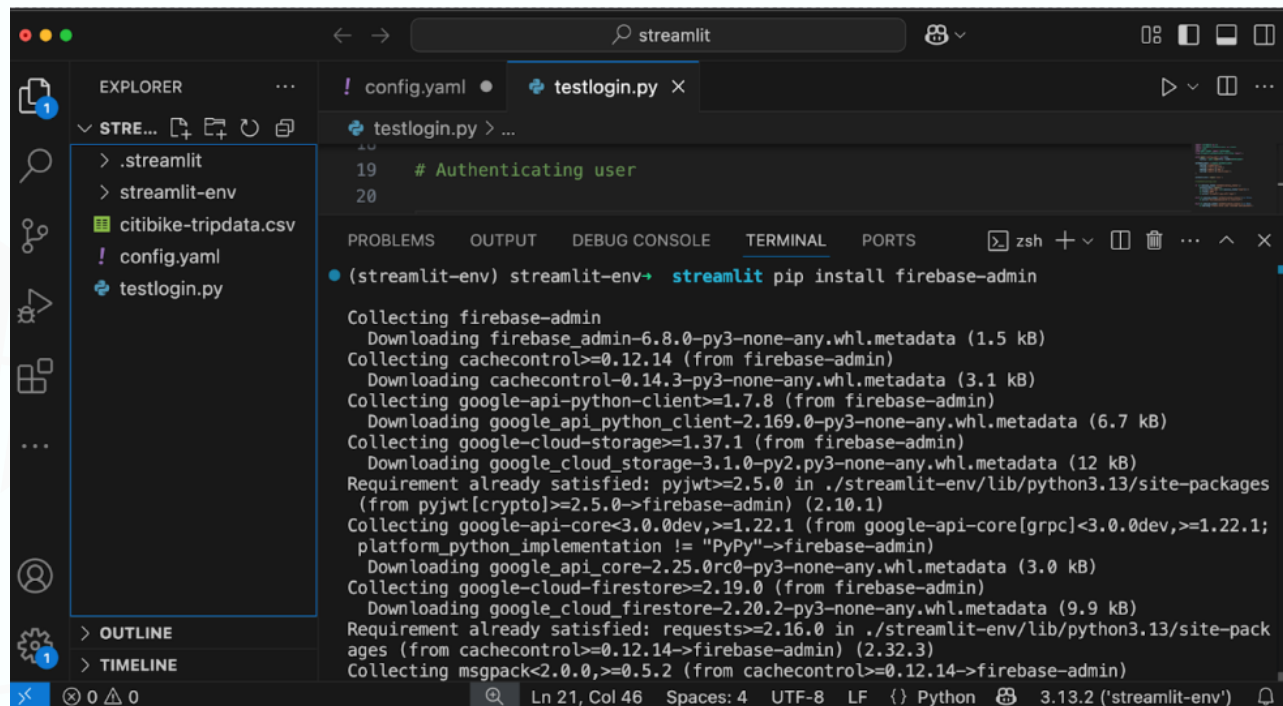


Paso 8. En **Visual Studio Code**, activa el ambiente virtual:



Paso 9. Instala el módulo de **Firebase** para el ambiente virtual:

```
$ pip install firebase-admin
```



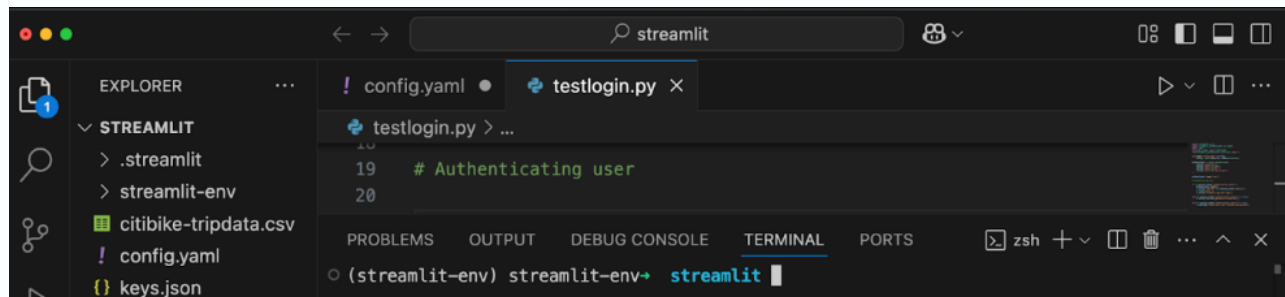
The screenshot shows a VS Code editor with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project structure with files like `config.yaml`, `testlogin.py`, and `streamlit-env`. The terminal window is titled `streamlit` and shows the command `streamlit pip install firebase-admin` being executed in a `streamlit-env` environment. The output of the command lists the packages being collected and downloaded, including `firebase-admin`, `cachecontrol`, `google-api-python-client`, `google-cloud-storage`, `google-api-core`, `google-cloud-firestore`, and `msgpack`.

```
config.yaml • testlogin.py x
testlogin.py > ...
19 # Authenticating user
20

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(streamlit-env) streamlit-env> streamlit pip install firebase-admin

Collecting firebase-admin
  Downloading firebase_admin-6.8.0-py3-none-any.whl.metadata (1.5 kB)
Collecting cachecontrol>=0.12.14 (from firebase-admin)
  Downloading cachecontrol-0.14.3-py3-none-any.whl.metadata (3.1 kB)
Collecting google-api-python-client>=1.7.8 (from firebase-admin)
  Downloading google_api_python_client-2.169.0-py3-none-any.whl.metadata (6.7 kB)
Collecting google-cloud-storage>=1.37.1 (from firebase-admin)
  Downloading google_cloud_storage-3.1.0-py2.py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: pyjwt>=2.5.0 in ./streamlit-env/lib/python3.13/site-packages
  (from pyjwt[crypto]>=2.5.0->firebase-admin) (2.10.1)
Collecting google-api-core<3.0.0dev,>=1.22.1 (from google-api-core[grpc]<3.0.0dev,>=1.22.1;
  platform_python_implementation != "PyPy"->firebase-admin)
  Downloading google_api_core-2.25.0rc0-py3-none-any.whl.metadata (3.0 kB)
Collecting google-cloud-firestore>=2.19.0 (from firebase-admin)
  Downloading google_cloud_firestore-2.20.2-py3-none-any.whl.metadata (9.9 kB)
Requirement already satisfied: requests>=2.16.0 in ./streamlit-env/lib/python3.13/site-pack
  ages (from cachecontrol>=0.12.14->firebase-admin) (2.32.3)
Collecting msgpack<2.0.0,>=0.5.2 (from cachecontrol>=0.12.14->firebase-admin)
  Downloading msgpack-1.0.8-py3-none-any.whl (125 kB)
Installing collected packages: msgpack, google-api-core, google-cloud-firestore, google-cloud-storage, google-api-python-client, cachecontrol, firebase-admin
Successfully installed cachecontrol-0.14.3 firebase-admin-6.8.0 google-api-core-2.25.0rc0 google-api-python-client-2.169.0 google-cloud-firestore-2.20.2 google-cloud-storage-3.1.0 msgpack-1.0.8
Ln 21, Col 46 Spaces: 4 UTF-8 LF {} Python 3.13.2 ('streamlit-env')
```

Paso 10. Renombra las llaves de **Firebase** a **keys.json**, y cópialas en la carpeta del proyecto:



Paso 11. Codifica un proceso para la inserción de los documentos **Firestore** con el siguiente algoritmo:

- Importar **Streamlit, Pandas y Firestore**.
- Leer las llaves del proyecto y crear una instancia.
- Crear una referencia hacia la colección **names** de **Firestore**.
- Crear controles de entrada para cada uno de los campos.
- Crear un control de botón.
- Determinar si el botón fue presionado con una sentencia `if`.
- Si fue presionado y además se llenaron los campos de entrada, crear una referencia al documento nuevo y usando el método `set()` insertar el nuevo registro.

Paso 12. En Visual Studio Code, crea demoFirestore.py:

```
import streamlit as st
import pandas as pd
from google.cloud import firestore
from google.oauth2 import service_account
db = firestore.Client.from_service_account_json("keys.
json")
dbNames = db.collection("names")

st.header("Nuevo registro")

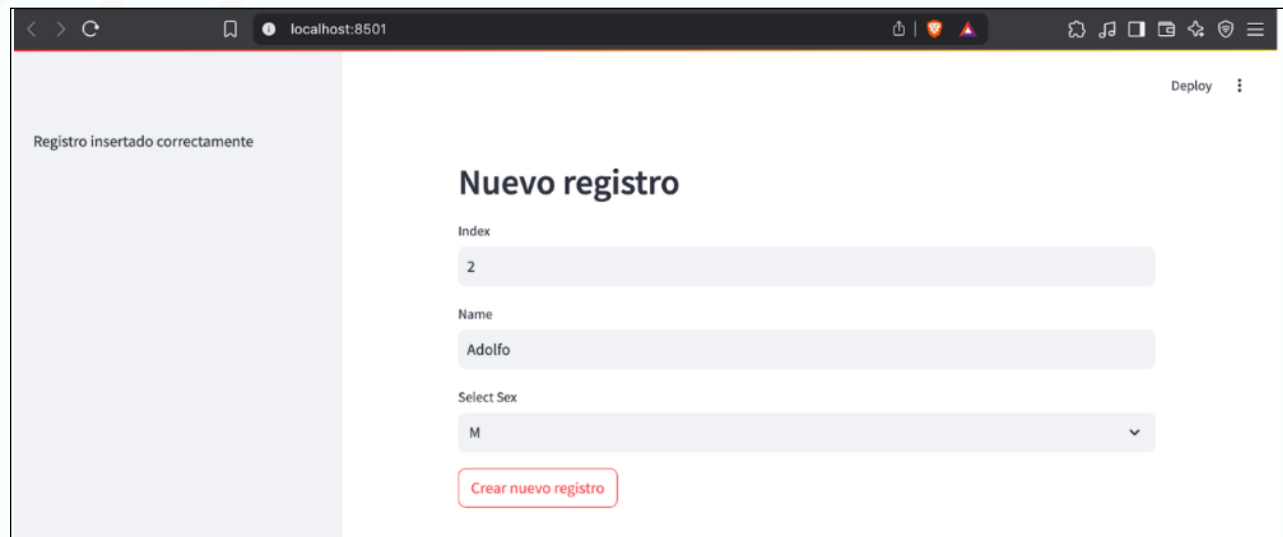
index = st.text_input("Index")
name = st.text_input("Name")
sex = st.selectbox(
    'Select Sex',
    ('F', 'M', 'Other'))

submit = st.button("Crear nuevo registro")

# Once the name has submitted, upload it to the
database
if index and name and sex and submit:
    doc_ref = db.collection("names").document(name)
    doc_ref.set({
        "index": index,
        "name": name,
        "sex": sex
    })
    st.sidebar.write("Registro insertado correctamente")
```

Paso 13. Ejecuta el proyecto con:

```
$ streamlit run demoFirestore.py
```



Registro insertado correctamente

Nuevo registro

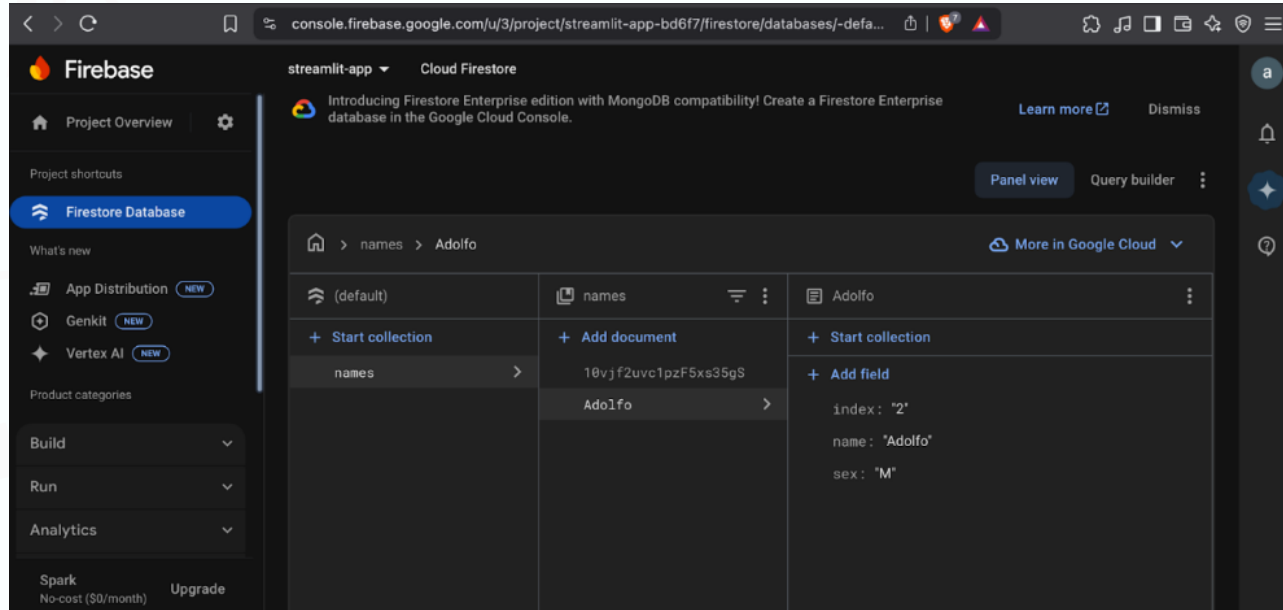
Index
2

Name
Adolfo

Select Sex
M

Crear nuevo registro

Paso 14. Comprueba los resultados de la inserción en **Firestore**:



Firestore

streamlit-app Cloud Firestore

Introducing Firestore Enterprise edition with MongoDB compatibility! Create a Firestore Enterprise database in the Google Cloud Console. [Learn more](#) [Dismiss](#)

Panel view Query builder

names > Adolfo

(default)	names	Adolfo
+ Start collection	+ Add document	+ Start collection
names >	10vjf2uvc1pzF5xs35gS Adolfo >	+ Add field
		index: "2" name: "Adolfo" sex: "M"

Después de la comprobación de resultados es momento de continuar con la operación de lectura de documentos.

Operación de lectura de los documentos

Ahora implementarás la operación de lectura de documentos de la colección de **Firestore**, para eso, continúa con los pasos:

Paso 15. Agrega el siguiente código a **demoFirestore.py**.

```
names_ref = list(db.collection(u'names').stream())

names_dict = list(map(lambda x: x.to_dict(), names_ref))

names_dataframe = pd.DataFrame(names_dict)

st.dataframe(names_dataframe)
```

Al hacerlo, está listo para continuar con otras operaciones.

Operaciones de eliminación y lectura

En este punto, implementarás las operaciones de eliminación y lectura en el proyecto. Continúa con los pasos:

Paso 16. Agrega el siguiente código:

```
# ...
def loadByName(name):
    names_ref = dbNames.where(u'name', u'==', name)
    currentName = None
    for myname in names_ref.stream():
        currentName = myname
    return currentName

st.sidebar.subheader("Buscar nombre")
nameSearch = st.sidebar.text_input("nombre")
btnFiltrar = st.sidebar.button("Buscar")
if btnFiltrar:
    doc = loadByName(nameSearch)
    if doc is None:
        st.sidebar.write("Nombre no existe")
    else:
        st.sidebar.write(doc.to_dict())
# ...

st.sidebar.markdown(""""---""")
btnEliminar = st.sidebar.button("Eliminar")

if btnEliminar:
    deletename = loadByName(nameSearch)
    if deletename is None:
        st.sidebar.write(f"{nameSearch} no existe")
    else:
```

```
dbNames.document(deletename.id).delete()
st.sidebar.write(f"{nameSearch} eliminado")
#...

st.sidebar.markdown("""---""")
newname = st.sidebar.text_input("Actualizar nombre")
btnActualizar = st.sidebar.button("Actualizar")

if btnActualizar:
    updatename = loadByName(nameSearch)
    if updatename is None:
        st.write(f"{nameSearch} no existe")
    else:
        myupdatename = dbNames.document(updatename.id)
        myupdatename.update(
            {
                "name": newname
            }
        )
```

Paso 17. Prueba nuevamente el proyecto con el código actualizado:

```
$ streamlit run demoFirestore.py
```

	sex	index	name
0	M	1	juan
1	M	2	Adolfo

Hasta ahora, has implementado un proyecto con **Streamlit** que realiza la conectividad a una base de datos no relacional en **Firestore** e implementa las operaciones básicas de lectura, inserción, eliminación y actualización.

Conexión con la base de datos no relacional: MongoDB

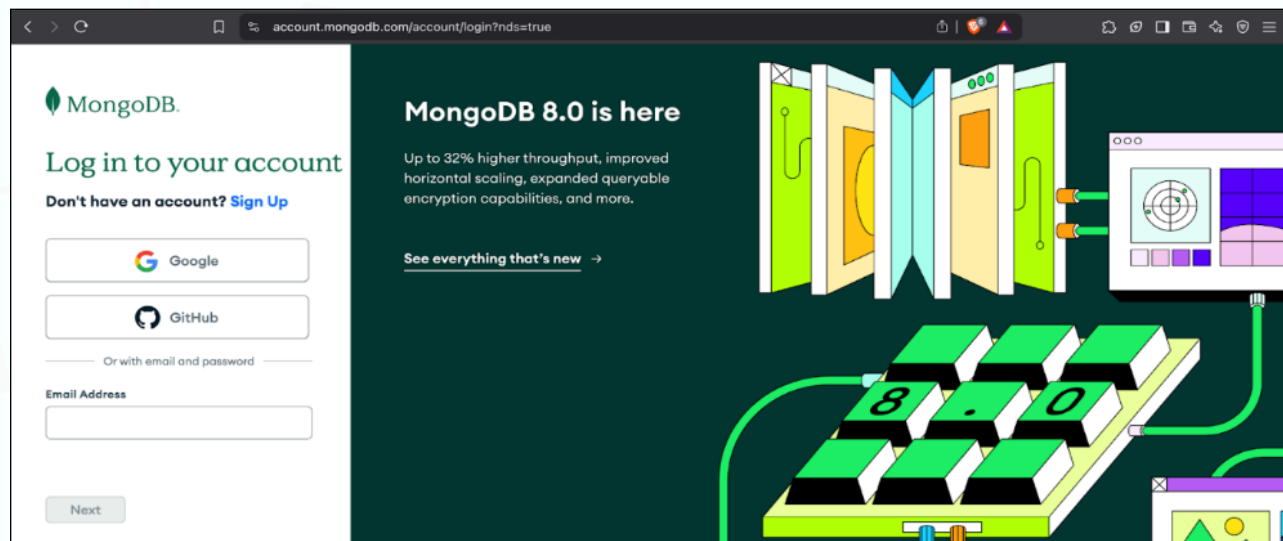
MongoDB es una base de datos de documentos de código abierto, desarrollada por MongoDB Inc, y lanzada en 2009. Principalmente, proporciona soporte para el almacenamiento de datos **JSON-like**. Al igual que la base de datos **NoSQL**, **MongoDB** almacena los datos en documentos **BSON (Binary JSON)**, permitiendo una gran flexibilidad en el esquema de los datos.

En esencia, **MongoDB** es conocida por su escalabilidad, flexibilidad y capacidad para manejar datos estructurados, semiestructurados y no estructurados a gran escala. **Por eso, para este proyecto usarás una versión en la nube llamada Atlas.** Para hacerlo, sigue estos pasos:

Paso 1. Crea una cuenta en el sitio oficial de **MongoDB**. Para esto, copia y pega el enlace en tu navegador: <https://account.mongodb.com/account/login?nds=true>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 12 de junio de 2025 y su disponibilidad depende del autor.

Puedes utilizar tu cuenta de **GitHub** para iniciar sesión.



Paso 2. Una vez dentro de **MongoDB Atlas**, crea un **clúster** y nómbralo **people**; asegúrate que sea la **opción gratuita**.

cloud.mongodb.com/v2/67d3b272439e8242a2b4afb8/clusters/starterTemplates

Deploy your cluster

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.

☒ **M30** **\$0.57/hour**
Dedicated cluster for high-traffic applications and large datasets.
STORAGE: 40 GB, RAM: 8 GB, vCPU: 2 vCPUs

☐ **M10** **\$0.08/hour**
Dedicated cluster for development environments and low-traffic applications.
STORAGE: 10 GB, RAM: 2 GB, vCPU: 2 vCPUs

☐ **Flex** **From \$0.011/hour**
Up to \$30/month
For application development and testing, with on-demand burst capacity for unpredictable traffic.
STORAGE: 6 GB, RAM: Shared, vCPU: Shared

Pay-as-you-go! You will be billed hourly and can terminate your cluster anytime. Excludes variable data transfer, backup, and taxes.

Configurations
Name: You cannot change the name once the cluster is created.
test
Provider: aws, Google Cloud, Azure
I'll do this later

Quick setup
☐ Preload sample dataset
Go to Advanced Configuration
Create Deployment

Al igual que en **Firestore**, puedes obtener la cadena de conexión al **clúster people** de **MongoDB**, para el lenguaje **Python** en la versión 3.0 o superior.

cloud.mongodb.com/v2/67d3b272439e8242a2b4afb8/overview?connectCluster=people

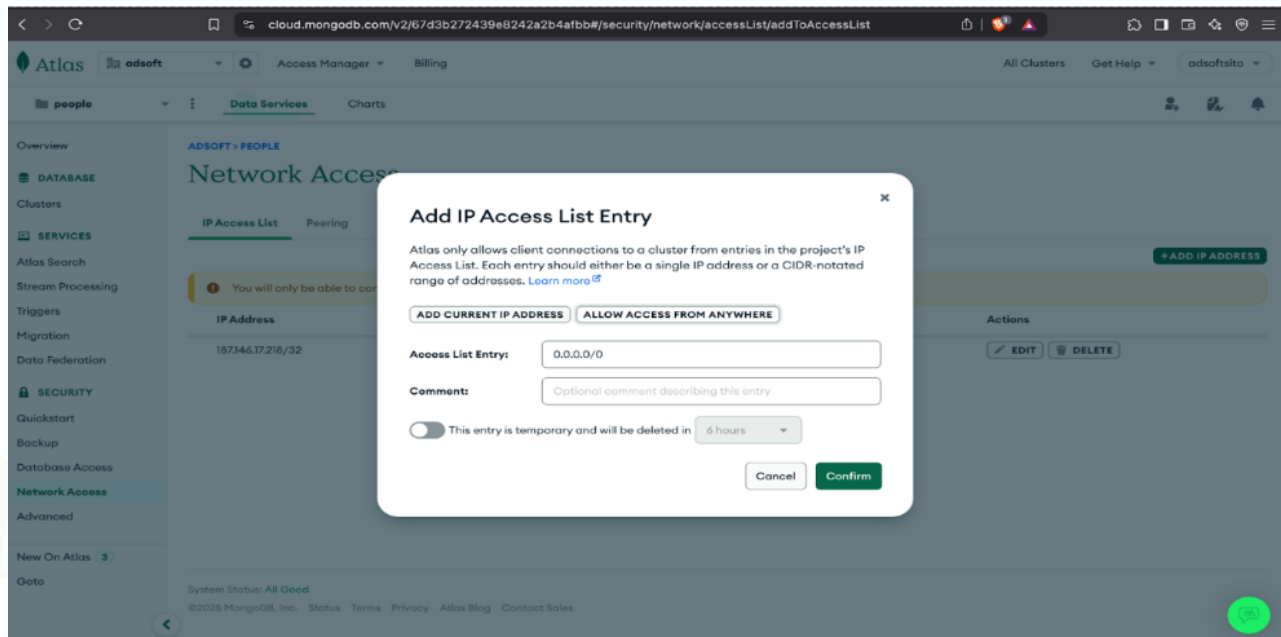
Connect to people

Set up connection security Choose a connection method **Connect**

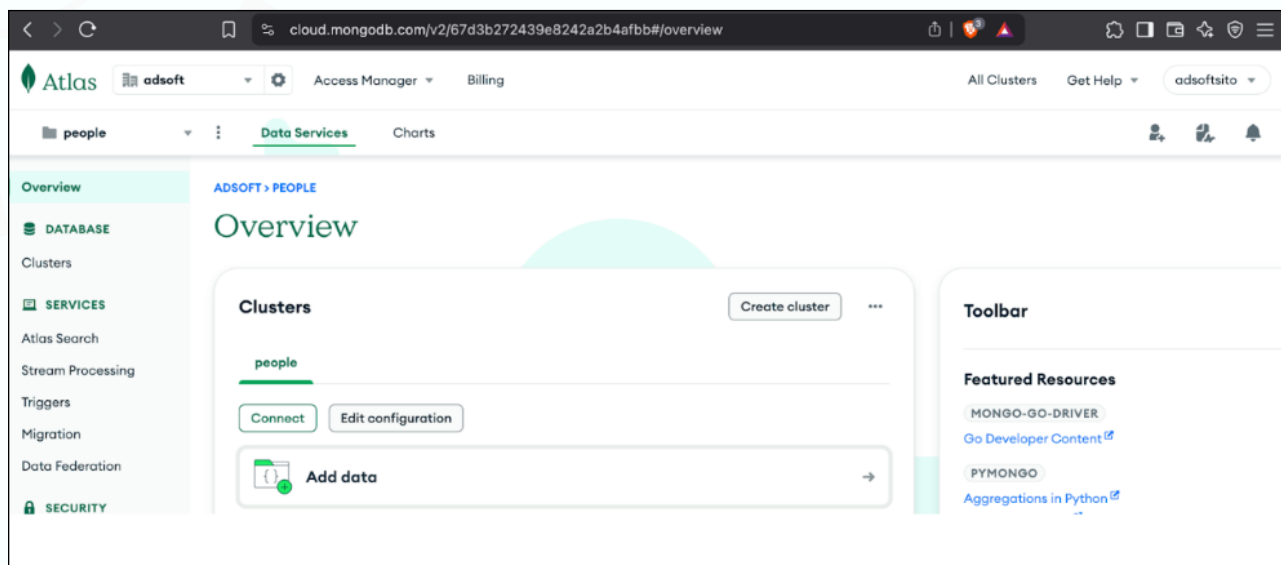
Connecting with MongoDB Driver

- Select your driver and version**
We recommend installing and using the latest driver version.
Driver: Python, Version: 3.11 or later
- Install your driver**
Run the following on the command line
Note: Use appropriate Python 3 executable
python -m pip install "pymongo[srv]"==3.11
View MongoDB Python Driver installation instructions.
- Add your connection string into your application code**
Use this connection string in your application
View full code sample
mongodb+srv://adsoft:<db_password>@people.1212d.mongodb.net/?retryWrites=true&w=majority&appName=people
Replace <db_password> with the password for the adsoft database user. Ensure any option params are URL encoded.
RESOURCES: Get started with the Python Driver, Python Driver Sample App

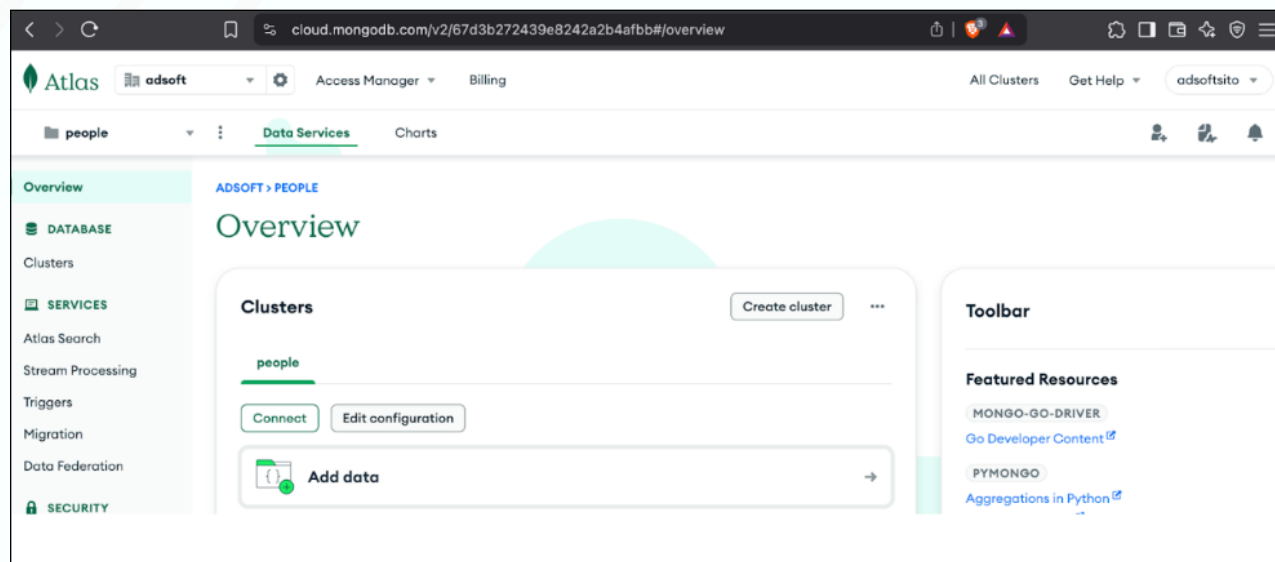
Paso 3. Configura el **firewall** del **clúster** para permitir el acceso externo desde la aplicación en **Python**. Para esto, selecciona la opción **ALLOW ACCESS FROM ANYWHERE**.



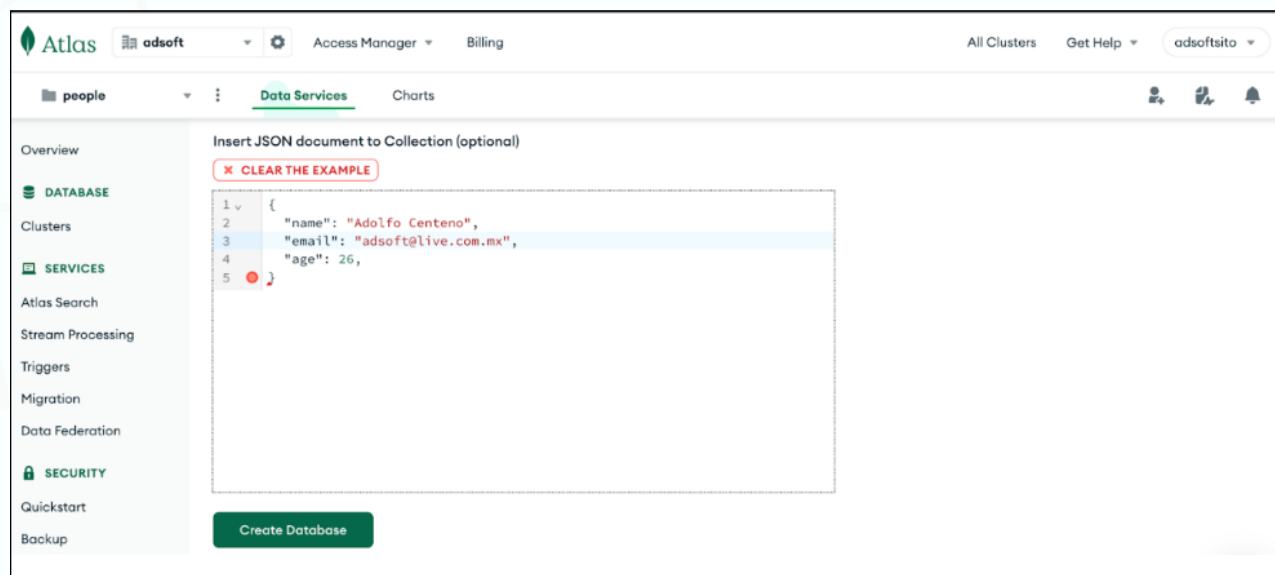
Paso 4. Al igual que en **Firestore**, en **MongoDB** debes crear colecciones para almacenar valores en estructuras llamadas documentos. Para hacerlo, da clic en la opción **Add Data**.



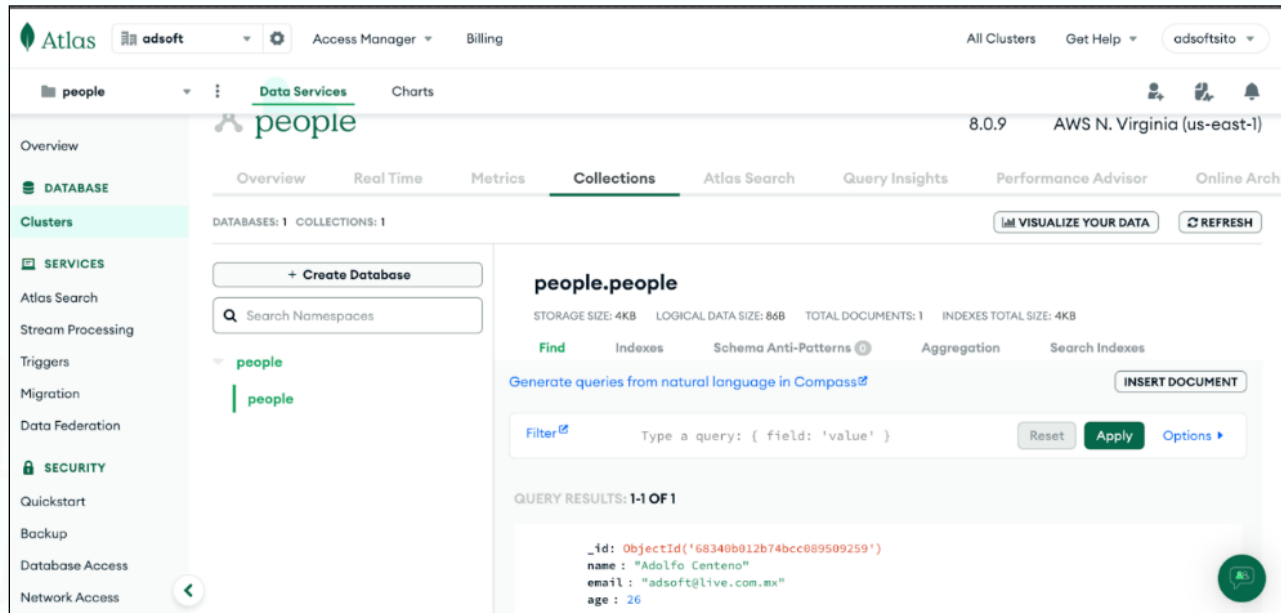
Paso 5. Crea una base de datos llamada **people**, y una colección llamada **people**.



Paso 6. Inserta un documento de ejemplo:

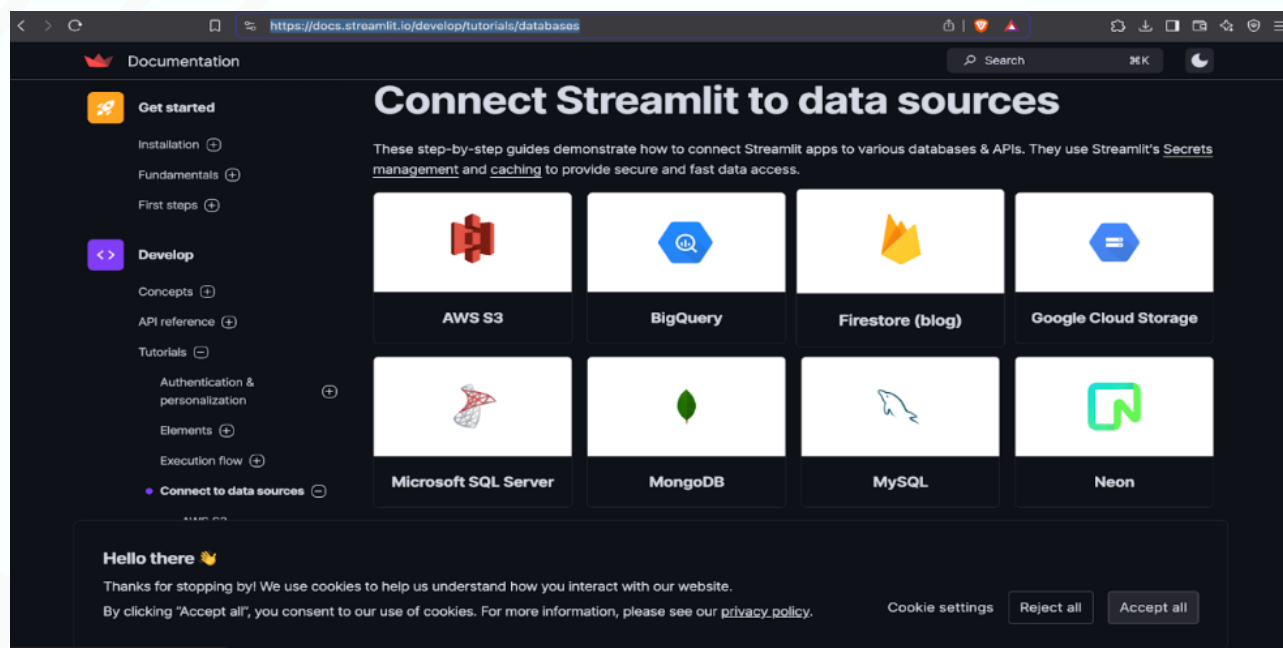


Paso 7. Da clic en **Create Database**, y observa cómo aparece la base de datos **people**, con la colección **people**, y el ejemplo de documento insertado.



Ahora, analiza la conexión desde **Python** a **MongoDB**. A modo de referencia, la página oficial de **Streamlit** contiene información detallada para la conexión con diversas fuentes de datos. Copia y pega el enlace en tu navegador para conocer más: <https://docs.streamlit.io/develop/tutorials/databases>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 12 de junio de 2025 y su disponibilidad depende del autor.



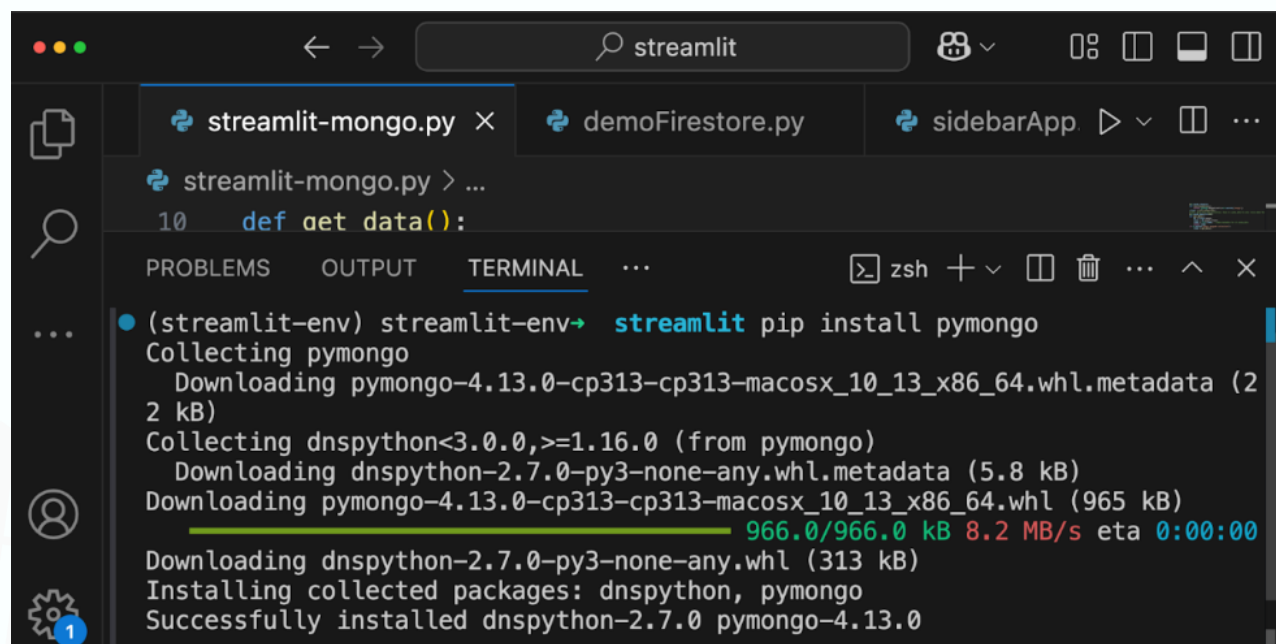
Particularmente, para obtener información sobre **MongoDB** y **Streamlit** puedes copiar y pegar en enlace en tu navegador: <https://docs.streamlit.io/develop/tutorials/databases/mongodb>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 12 de junio de 2025 y su disponibilidad depende del autor.

Es momento de implementar la conectividad en el proyecto de **Firebase**. Para esto, continúa con los pasos:

Paso 8. En **Visual Studio Code**, activa el ambiente virtual e instala el *driver* de **MongoDB** para **Python**.

```
$ source streamlit-env/bin/activate
$ pip install pymongo
```



The screenshot shows the Visual Studio Code interface with a terminal window open. The terminal is running in a virtual environment named 'streamlit-env'. The user has entered the command 'pip install pymongo'. The terminal output shows the following steps: collecting pymongo, downloading pymongo-4.13.0-cp313-cp313-macosx_10_13_x86_64.whl.metadata (2 kB), collecting dnspython<3.0.0,>=1.16.0 (from pymongo), downloading dnspython-2.7.0-py3-none-any.whl.metadata (5.8 kB), downloading pymongo-4.13.0-cp313-cp313-macosx_10_13_x86_64.whl (965 kB), downloading dnspython-2.7.0-py3-none-any.whl (313 kB), installing collected packages: dnspython, pymongo, and successfully installing dnspython-2.7.0 and pymongo-4.13.0. The progress bar for pymongo shows 966.0/966.0 kB at 8.2 MB/s with an eta of 0:00:00.

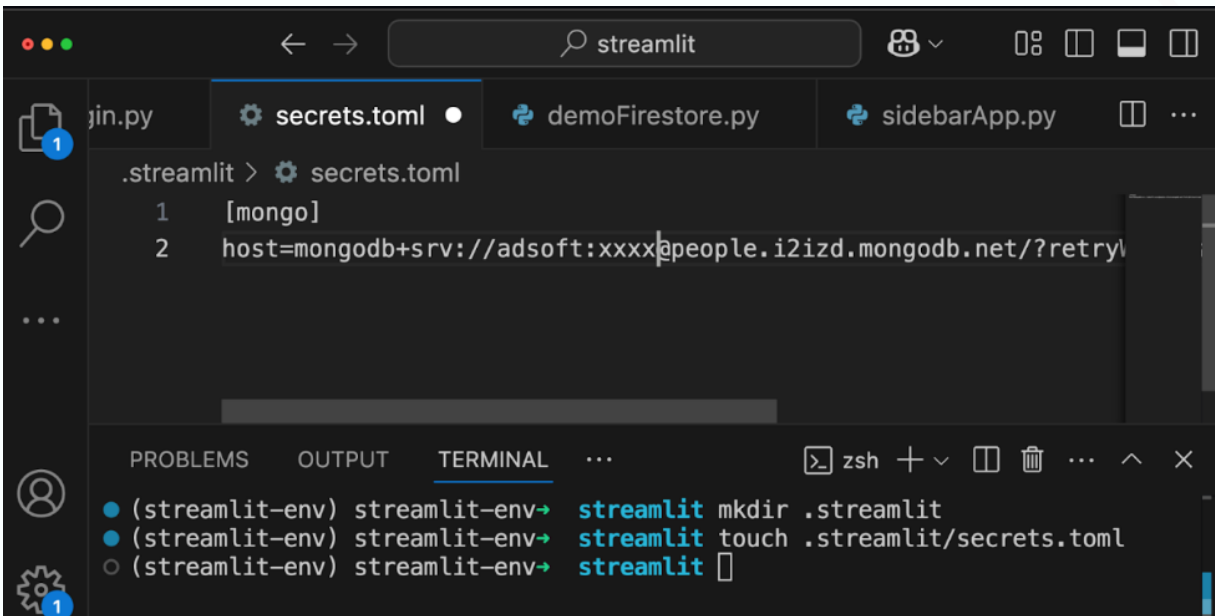
```
(streamlit-env) streamlit-env➔ streamlit pip install pymongo
Collecting pymongo
  Downloading pymongo-4.13.0-cp313-cp313-macosx_10_13_x86_64.whl.metadata (2
2 kB)
Collecting dnspython<3.0.0,>=1.16.0 (from pymongo)
  Downloading dnspython-2.7.0-py3-none-any.whl.metadata (5.8 kB)
  Downloading pymongo-4.13.0-cp313-cp313-macosx_10_13_x86_64.whl (965 kB)
  Downloading dnspython-2.7.0-py3-none-any.whl (313 kB)
Installing collected packages: dnspython, pymongo
Successfully installed dnspython-2.7.0 pymongo-4.13.0
```

Paso 9. Crea el archivo con las credenciales para la base de datos de **Atlas MongoDB**.

```
$ mkdir .streamlit
$ touch .streamlit/secrets.toml
```

Paso 10. Edita `.streamlit/secrets.toml` con la configuración de la base de datos de **MongoDB Atlas**, por ejemplo:

```
1 [mongo]
2 host = "people.i2izd.mongodb.net/?retryWrites=true&w=majority&appName=people"
3 port = 27017
4 username = "adsoft"
5 password = "XXXXX"
```



Paso 11. En Visual Studio Code, crea streamlit-mongo.py:

```
$ touch streamlit-mongo.py
```

Paso 12. Edita streamlit-mongo.py con el siguiente código:

```
import streamlit as st
import pymongo
# Initialize connection. Uses st.cache_resource to
only run once.
@st.cache_resource
def init_connection():
    return pymongo.MongoClient(**st.secrets["mongo"])
client = init_connection()
# Pull data from the collection. Uses st.cache_data
to only rerun when the query changes or after 10 min.
@st.cache_data(ttl=600)
def get_data():
    db = client.people
    items = db.people.find()
```

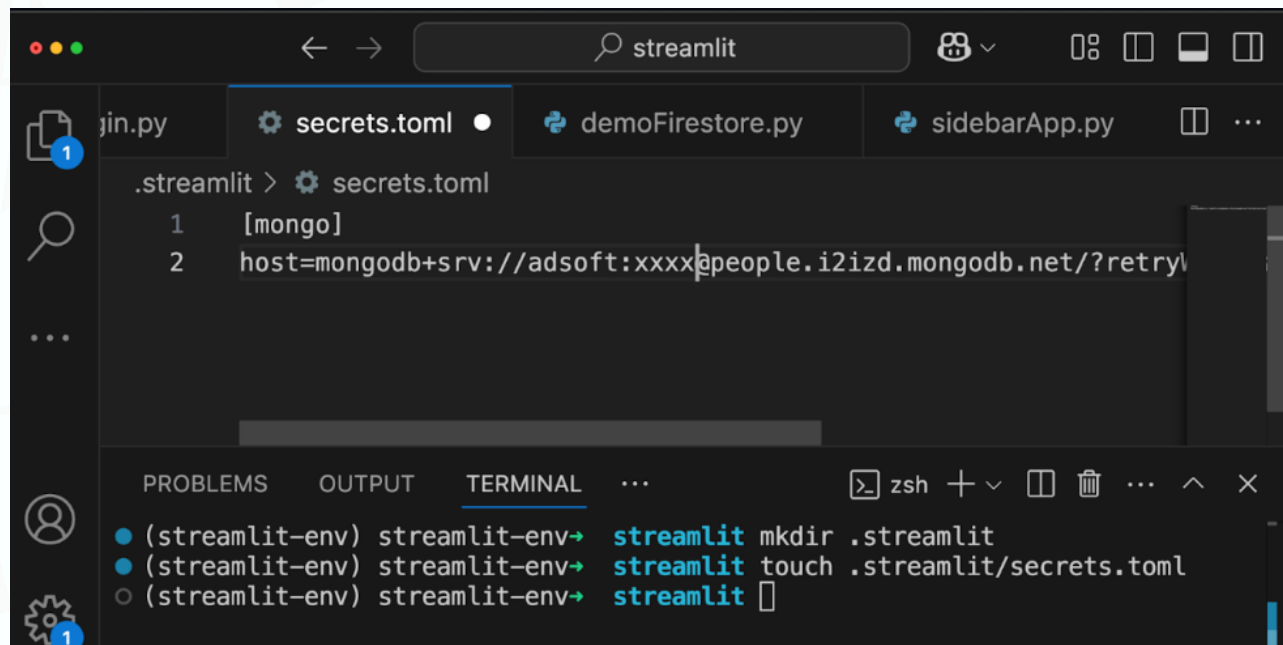
```

        items = list(items)    # make hashable for st.cache_data
    return items
if st.button("Query mongodb collection"):
    items = get_data()
    # Print results.
    for item in items:
        st.write(item)

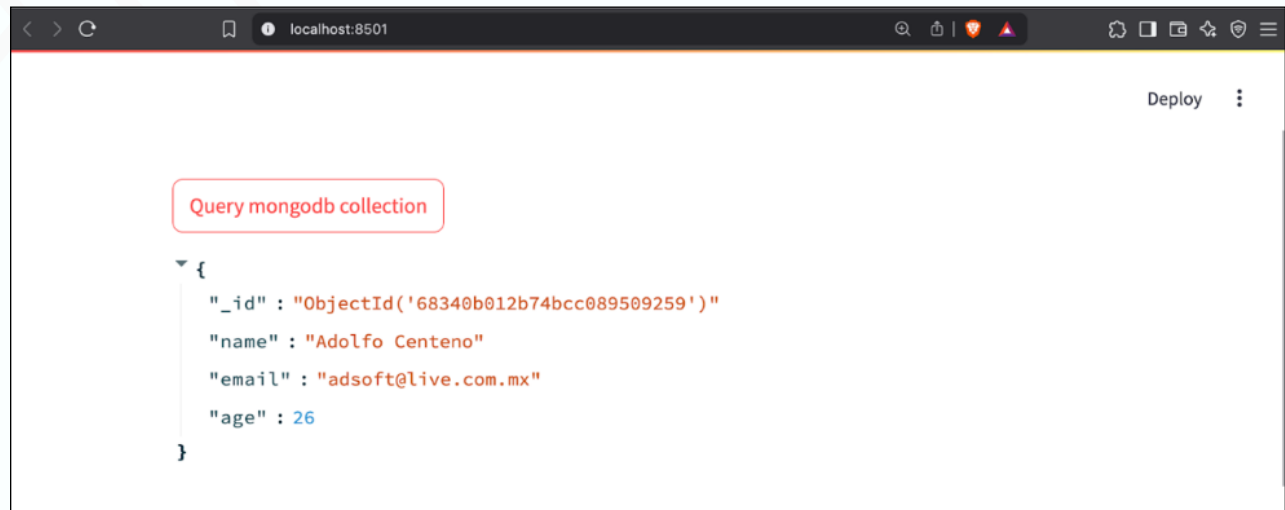
```

Paso 13. Prueba la aplicación con:

```
$ streamlit run streamlit-mongo.py
```



Finalmente, observa cómo en el navegador se muestra la colección recuperada de **MongoDB Atlas**.



Hasta aquí, revisaste el consumo de datos de **Firestore** y **MongoDB**, continúa para conocer más sobre bases de datos relacionales con **PostgreSQL**.

Pruébate

Ejercicio de reflexión

El conocimiento de las operaciones básicas en base de datos no relacional como **Firestore** o **MongoDB** desde **Streamlit** son de vital importancia para insertar información de negocios en cualquier proyecto de *software*. Ahora que has estudiado cómo insertar datos de forma permanente en una base de datos de **Firestore** y **MongoDB**, reflexiona los siguientes aspectos:

- ¿Cuál es la importancia de analizar la información y estructura de los documentos que quieres almacenar en **Firestore** o **MongoDB**?

- ¿Qué otros tipos de operaciones de negocios puedes automatizar almacenando sus datos en **Firestore** o **MongoDB**?
- Si alguna vez has hecho una aplicación con bases de datos en otra tecnología, ¿qué grado de dificultad encuentras integrando **Streamlit** y **Firestore/MongoDB**?

Cuestionario

Instrucciones: Lee con atención cada pregunta y selecciona una respuesta.

1. ¿Cuál es la sintaxis correcta para establecer la referencia a **un solo documento** dentro de una colección?

a. `doc_ref = db.collection("mycollection")`

b. `doc_ref = db.collection("mycollection").document("myDocumentId")`

c. `doc_ref = db.document("myDocumentId").collection("mycollection")`

Retroalimentación: La sintaxis donde se encuentra la colección seguida del Id del documento es: `doc_ref = db.collection("mycollection").document("myDocumentId")`

2. Si hemos creado una referencia llamada `doc_ref`, ¿cuál es la sintaxis correcta para crear un nuevo documento?

```
a. doc_ref.set({  
    "campo1": valor1,  
    "campo2": valor2,  
    "campon": valor1n  
})
```

```
b. doc_ref.update({  
    "campo1": valor1,  
    "campo2": valor2,  
    "campon": valor1n  
})
```

```
c. doc_ref.delete()
```

Retroalimentación: El método `set()` crea un nuevo documento especificado en formato **JSON**. Por lo tanto, la sintaxis adecuada es:

```
doc_ref.set({  
    "campo1": valor1,  
    "campo2": valor2,  
    "campon": valor1n  
})
```

Consumo de base de datos relacionales con PostgreSQL

Una base de **datos relacional** es un tipo de base de datos que organiza la información en tablas conectadas entre sí mediante relaciones. **Estas tablas tienen filas (registros) y columnas (atributos)**, que permiten estructurar y consultar datos de manera eficiente. En este modelo, las relaciones entre las tablas se definen a través de claves primarias y foráneas, permitiendo establecer conexiones lógicas entre diferentes conjuntos de datos. Sin embargo, tienen limitaciones en cuanto a la escalabilidad horizontal y la flexibilidad para manejar datos no estructurados.

Las bases de datos relacionales utilizan el **Structured Query Language (SQL) o Lenguaje de Consulta Estructurado**, para almacenar, manipular y recuperar datos. Algunos ejemplos de sistemas manejadores de base de datos relacionales son:

- **MySQL**: Es un sistema de gestión de bases de datos relacional ampliamente utilizado y de código abierto.
- **PostgreSQL**: Ofrece funcionalidades avanzadas y es conocido por su cumplimiento riguroso de los estándares **SQL**.
- **Microsoft SQL Server**: Proporciona herramientas robustas para la gestión de datos.

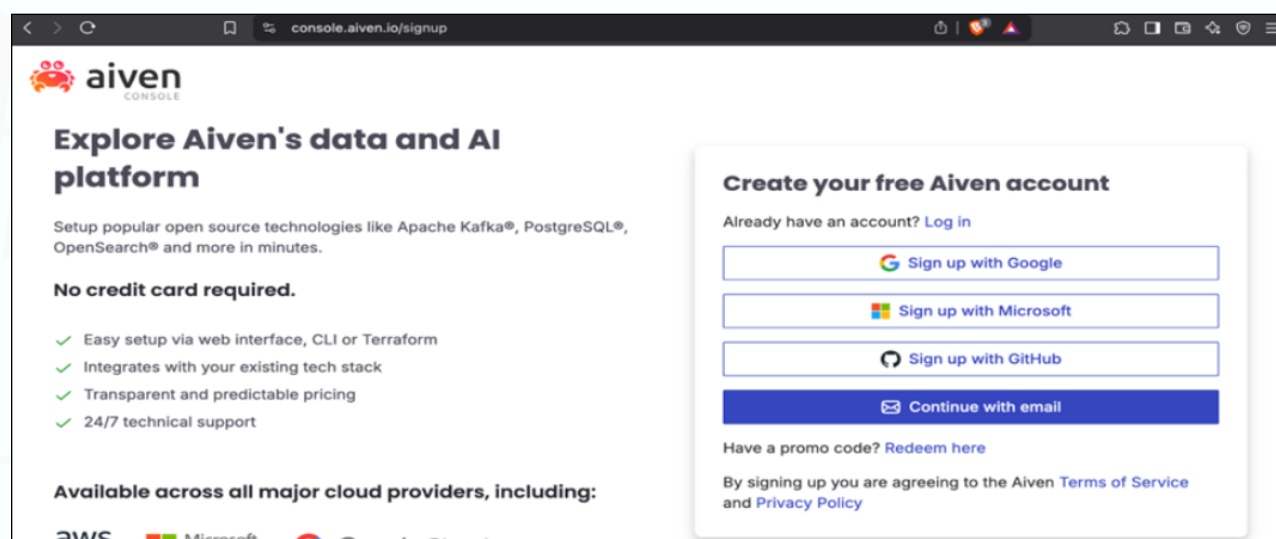
Estas bases de datos son comunes en aplicaciones financieras, sistemas de gestión de inventarios y aplicaciones empresariales debido a su capacidad para mantener la integridad y consistencia de los datos.

Creación de una database PostgreSQL

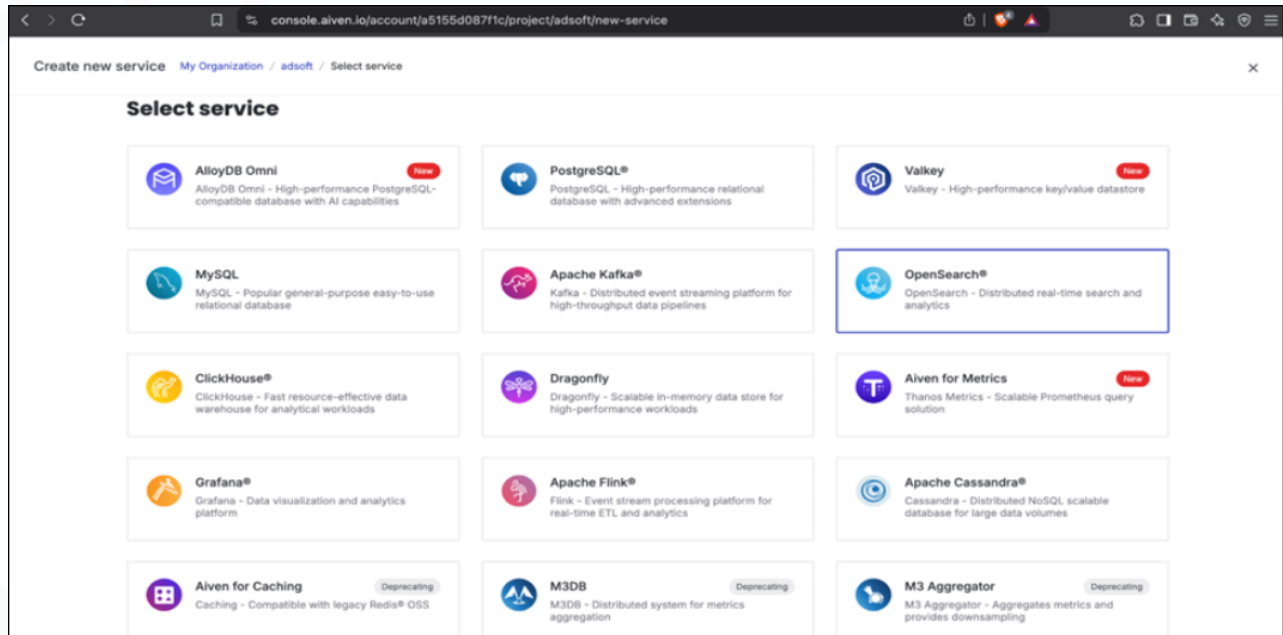
En este momento, crearás una base de datos **PostgreSQL** en la nube, con el propósito de administrarla y posteriormente consumirla desde **Python** y **Streamlit**. Para hacerlo, sigue estos pasos:

Paso 1. Utiliza tu cuenta de GitHub o de Gmail, y crea una cuenta en: <https://console.aiven.io/signup>. Copia y pega el enlace en tu navegador para acceder.

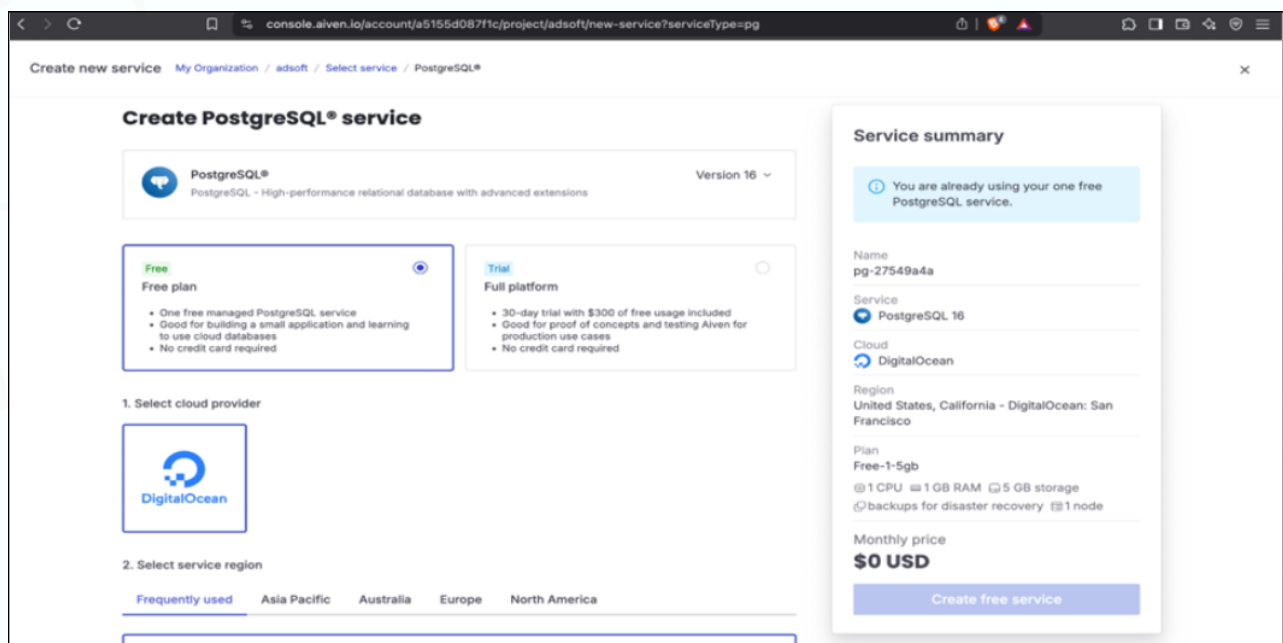
Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 12 de junio de 2025 y su disponibilidad depende del autor.



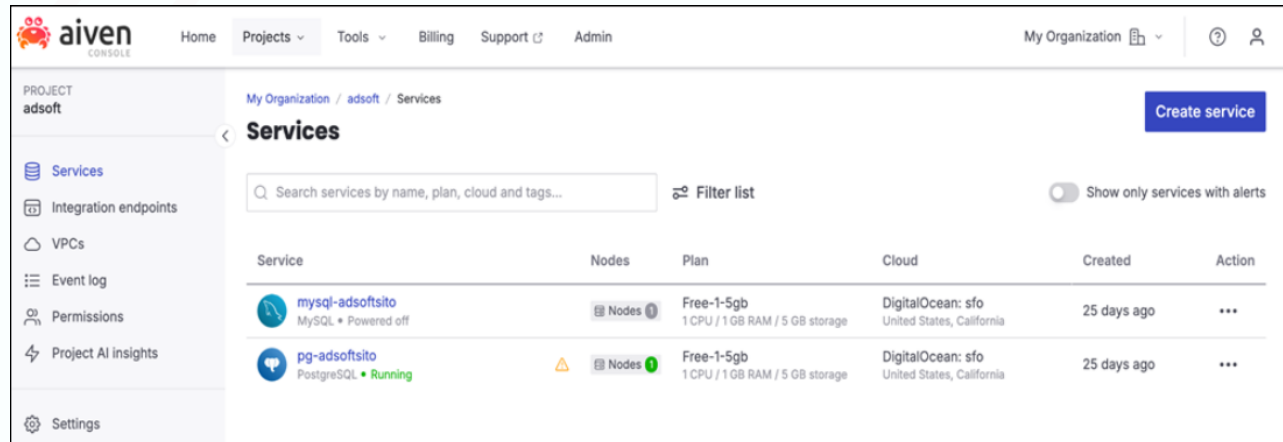
Paso 2. Selecciona la base de datos **PostgreSQL**, otra buena opción es **MySQL**.



Paso 3. Selecciona la capa gratuita para el servicio.



Paso 4. Una vez creado el servidor, observa cómo luce el *dashboard*. En este caso, existen dos servidores: uno de **MySQL** y otro de **PostgreSQL**.

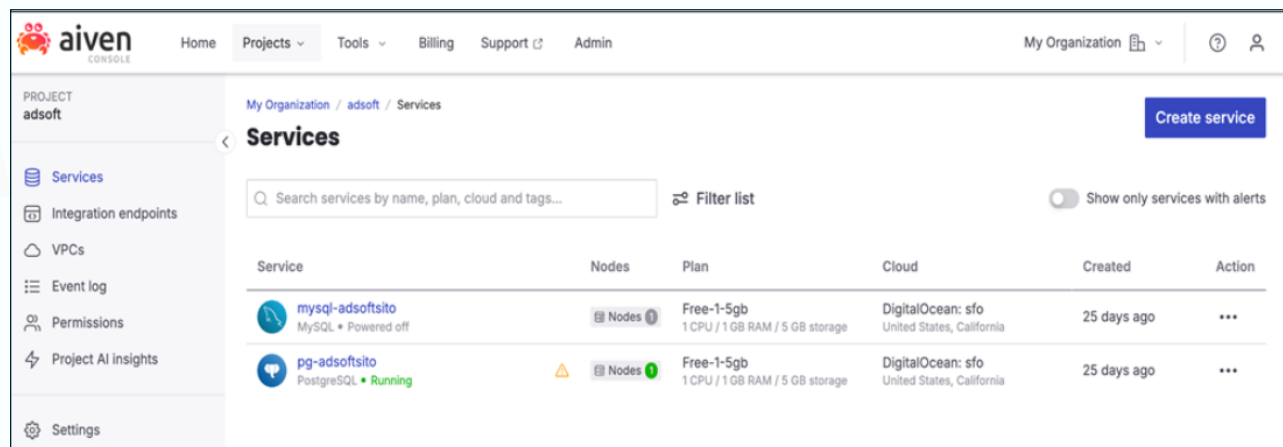


The screenshot shows the Aiven console dashboard. The left sidebar contains navigation links: Services, Integration endpoints, VPCs, Event log, Permissions, Project AI insights, and Settings. The main content area is titled 'Services' and shows a list of two services:

Service	Nodes	Plan	Cloud	Created	Action
mysql-adsoftito MySQL • Powered off	Nodes 1	Free-1-5gb 1 CPU / 1 GB RAM / 5 GB storage	DigitalOcean: sfo United States, California	25 days ago	...
pg-adsoftito PostgreSQL • Running	Nodes 1	Free-1-5gb 1 CPU / 1 GB RAM / 5 GB storage	DigitalOcean: sfo United States, California	25 days ago	...

Paso 5. Selecciona la instancia de **PostgreSQL** para obtener las credenciales de acceso, copia el nombre del **Host**, base de datos, nombre de usuario y contraseña, y el puerto. Estos datos serán utilizados para la instalación de **pgAdmin**.

Nota: El puerto por defecto para PostgreSQL es 5432, aunque en esta nube regularmente, esta asignado el 13078.



This is a duplicate of the screenshot above, showing the Aiven console dashboard with two services: mysql-adsoftito and pg-adsoftito.

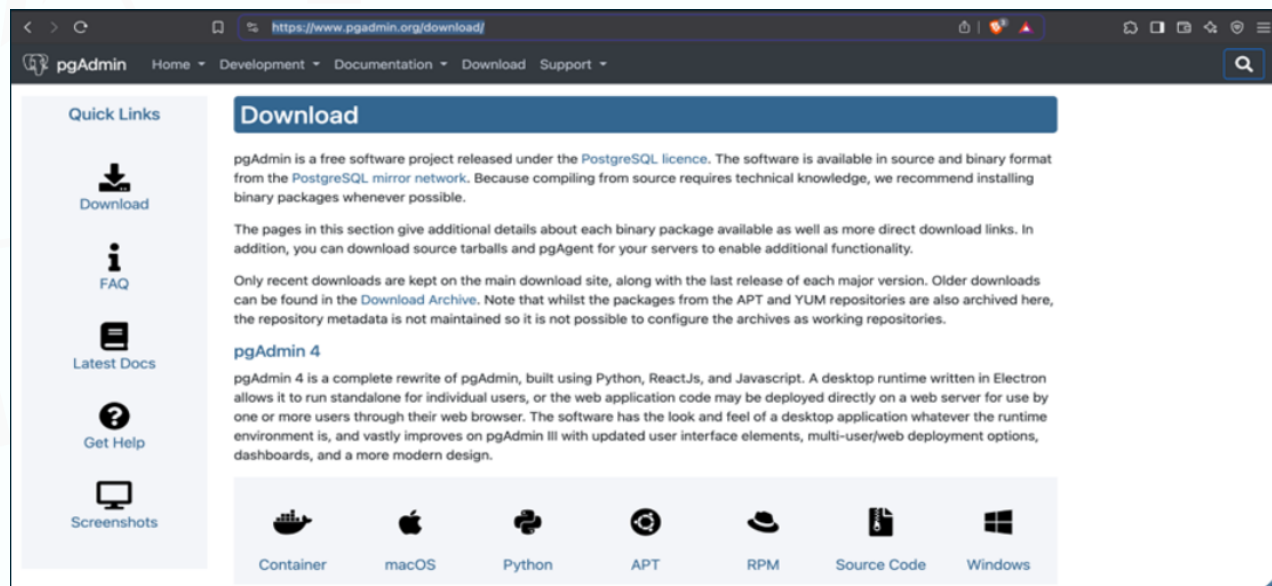
Instalación de cliente pgAdmin

Por la naturaleza del servidor, es decir, que está en la nube, necesitas instalar de forma local un cliente para administrar la base de datos. Por eso, instalarás **pgAdmin**, que puedes encontrar en: <https://www.pgadmin.org/download/>. Copia y pega el enlace en tu navegador para acceder.

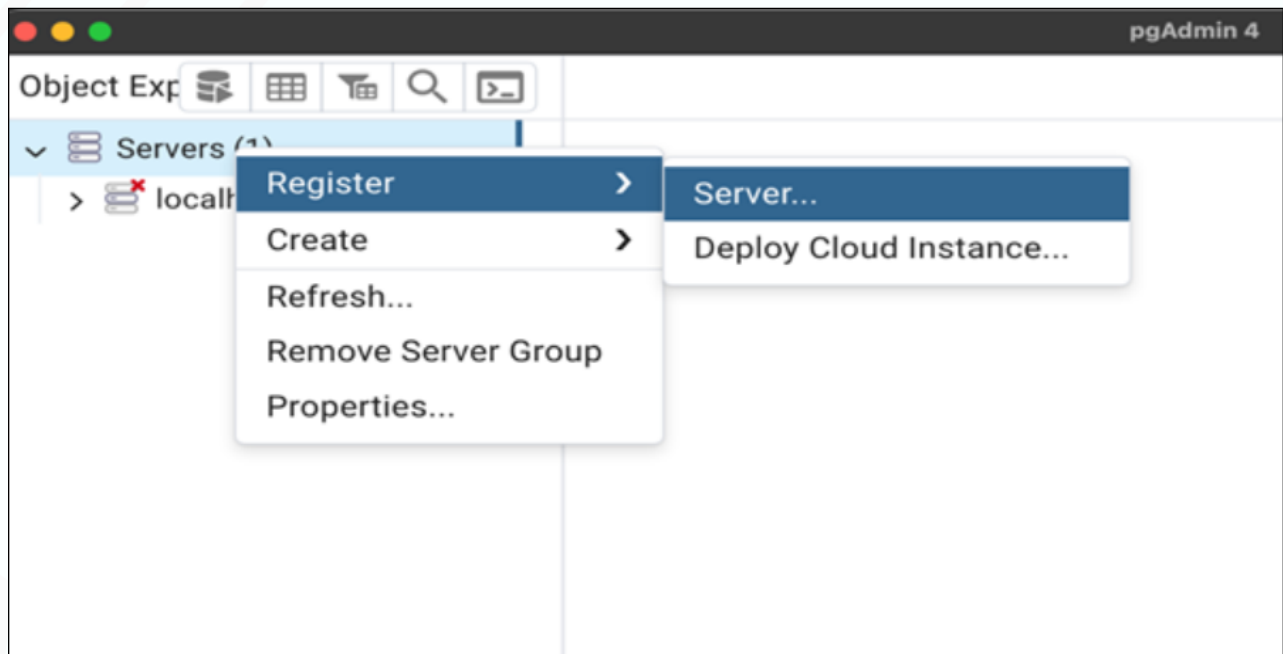
Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 12 de junio de 2025 y su disponibilidad depende del autor.

Para hacerlo, sigue estos pasos:

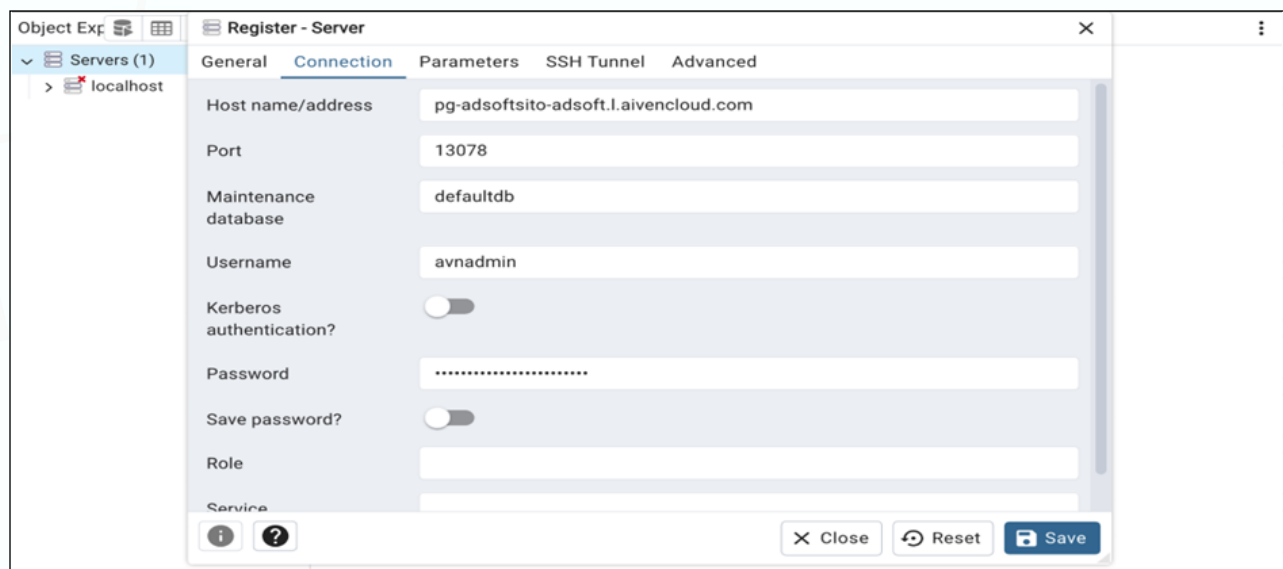
Paso 1. Selecciona el paquete de instalación adecuado para tu sistema operativo.



Paso 2. Una vez instalado **pgAdmin**, ábrelo para registrar las credenciales del servidor en la nube.

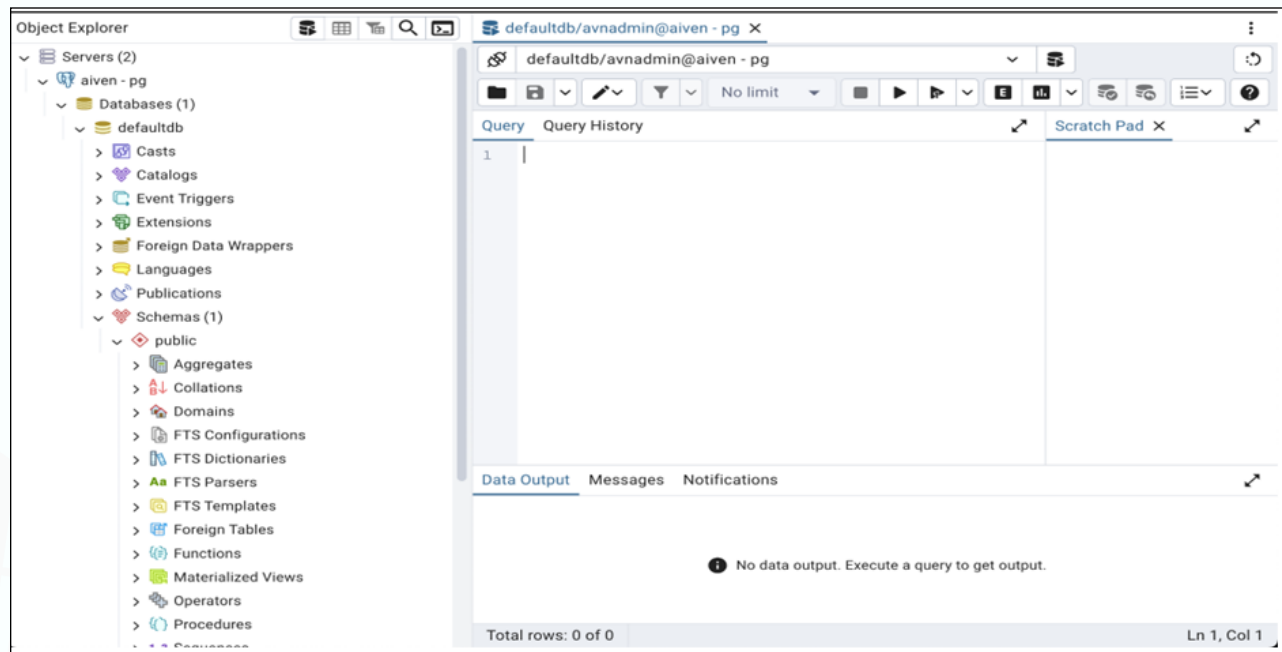


Paso 3. Selecciona la pestaña de **Connection**, e ingresa las credenciales de nube de **aiven**.

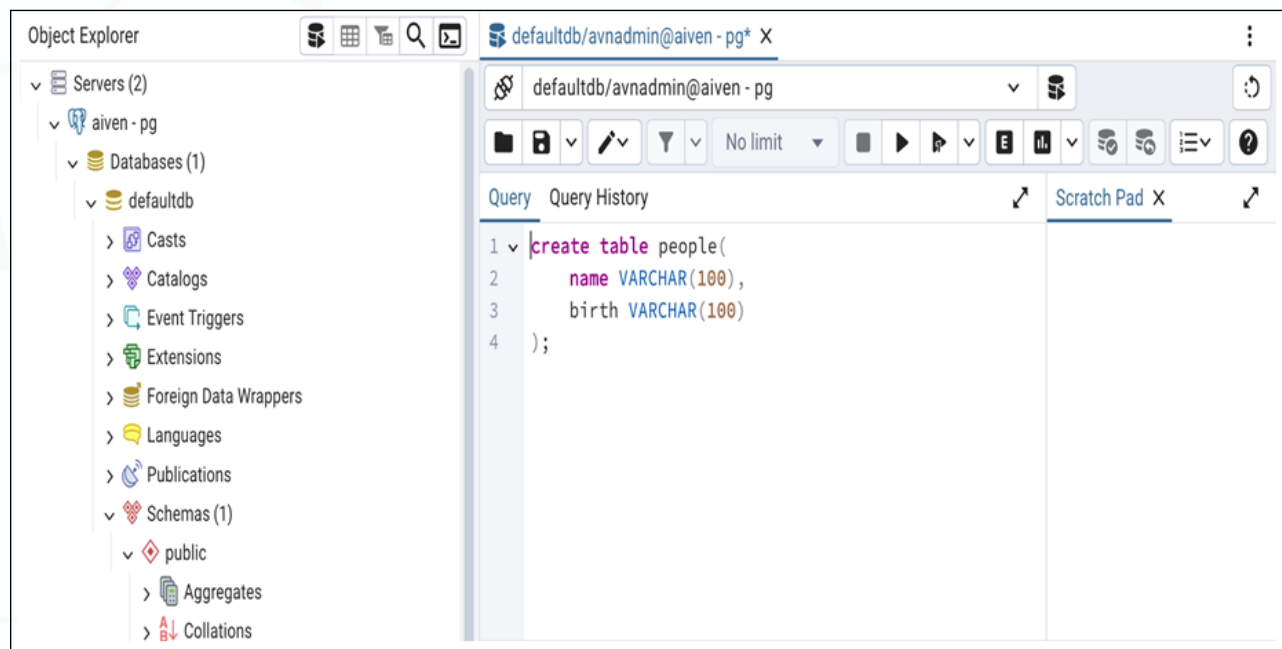


Paso 4. Una vez realizada la conexión a la base de datos de la nube de **aiven**, crea una tabla para guardar algunos datos que consultarás de la aplicación en **Streamlit**.

Paso 5. Abre la aplicación **Query Tool**, que permite ingresar comandos en **SQL** para crear tablas, insertar, eliminar, consultar y actualizar datos.



Paso 6. Ingresa el comando **SQL** para crear una tabla llamada **people**, con dos campos: **name** y **birth**.

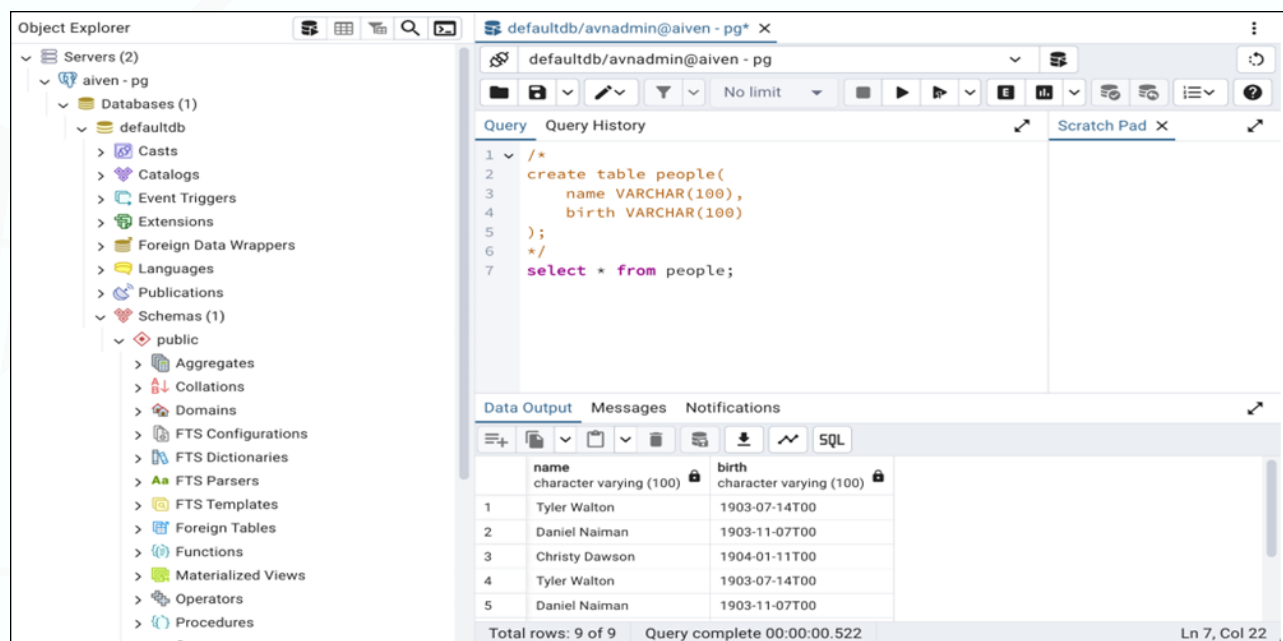


Paso 7. Inserta algunos datos de prueba en la tabla **people**, con el comando **SQL**:

```
insert into people(name, birth) values ("Adolfo",  
"21-10-1976");  
insert into people(name, birth) values ("Miriam",  
"21-09-1976");
```

Paso 8. Análogamente, visualiza el contenido de la tabla **people** con el comando **SQL**:

```
select * from people;
```



The screenshot shows the pgAdmin interface. On the left, the Object Explorer shows the database structure. The main pane displays the SQL editor with the following code:

```
1 /*  
2 create table people(  
3     name VARCHAR(100),  
4     birth VARCHAR(100)  
5 );  
6 */  
7 select * from people;
```

Below the SQL editor, the Data Output pane shows the results of the query:

	name character varying (100)	birth character varying (100)
1	Tyler Walton	1903-07-14T00
2	Daniel Naiman	1903-11-07T00
3	Christy Dawson	1904-01-11T00
4	Tyler Walton	1903-07-14T00
5	Daniel Naiman	1903-11-07T00

Total rows: 9 of 9 Query complete 00:00:00.522 Ln 7, Col 22

En este punto, crearás la aplicación en **Streamlit** y **Python** para consumir la base de datos de **PostgreSQL**. Continúa con los pasos:

Paso 9. En **Visual Studio Code**, actualiza el archivo **.streamlit/secrets.toml** para agregar la conexión con las credenciales de **PostgreSQL**.

```
# .streamlit/secrets.toml

[connections.postgresql]
dialect = "postgresql"
host = "localhost"
port = "5432"
database = "xxx"
username = "xxx"
password = "xxx"
```

Paso 10. En **Visual Studio Code**, crea el cliente en **Python streamlit-pg.py** para consumir la base de datos de **PostgreSQL**, con el siguiente código:

```
import streamlit as st
conn = st.connection("postgresql", type="sql")
if st.button("Query Postgresql table"):
    # Perform query.
    df = conn.query('SELECT * FROM people;', ttl="10m")
    # Print results.
    for row in df.itertuples():
        st.write(row)
```

Paso 11. Finalmente, ejecuta la aplicación con: `streamlit run streamlit-pg.py`

Pruébate

Ejercicio de reflexión

Las habilidades en la administración de base de datos relacionales como **MySQL** o **PostgreSQL** son altamente valoradas en la gran mayoría de proyectos de software. Ahora que has estudiado cómo crear y administrar base de datos relacionales, reflexiona los siguientes aspectos:

- ¿Cuál es la importancia de analizar la información y estructura de los documentos que quieres almacenar en una base de datos relacional
- ¿Qué tipo de operaciones de negocios puedes automatizar usando una base de datos relacional?
- Si alguna vez has hecho una aplicación con bases de datos en otra tecnología, ¿qué grado de dificultad encuentras integrando **Streamlit** y **PostgreSQL**?

Cuestionario

Instrucciones: Lee con atención cada pregunta y selecciona una respuesta.

En **PostgreSQL**, ¿cuál es puerto por *default* donde escucha este servidor de base de datos?

- a. 5432
- b. 8080
- c. 8501

Retroalimentación: El servidor de base de datos **PostgreSQL** escucha por *default* en el puerto 5432.

2. ¿Cuál es el comando **SQL** adecuado para crear una tabla llamada **alumnos** con los campos **matrícula** y **nombre**?

a. `create table alumnos(
 VARCHAR(10) matricula,
 VARCHAR(50) nombre
);`

b. `create table alumnos(
 matricula VARCHAR(10),
 nombre VARCHAR(50)
);`

c. `create table alumnos(
 matricula,
 nombre
);`

Retroalimentación: La sintaxis correcta para crear la tabla llamada **alumnos** con los campos **matrícula** y **nombre** es:

```
create table alumnos(  
    matricula VARCHAR(10),  
    nombre VARCHAR(50)  
);
```

3. ¿Cuál es el comando **SQL** adecuado para insertar un registro nuevo en una tabla llamada **clientes** con campos **rfc**, **razon_social** y **dirección**?

```
a. insert into clientes(rfc, razon_social, direccion)
values ("CETA7610219C9","ADOLFO CENTENO","LERDO DE
TEJADA 107, ORIZABA, VERACRUZ");
```

```
b. insert into clientes(rfc, razon_social, direccion)
values ("ADOLFO CENTENO","LERDO DE TEJADA 107, ORIZA-
BA, VERACRUZ");
```

```
c. insert into clientes() values ("CETA7610219C9","ADOL-
FO CENTENO","LERDO DE TEJADA 107, ORIZABA, VERACRUZ");
```

Retroalimentación: El comando **SQL** que inserta correctamente un nuevo registro en la tabla **clientes**, con los campos: **rfc**, **razon_social** y **direccion** es:

```
insert into clientes(rfc, razon_social, direccion)
values ("CETA7610219C9","ADOLFO CENTENO","LERDO DE
TEJADA 107, ORIZABA, VERACRUZ");
```

Ideas para llevar

- Identificaste que **Firebase** y **Streamlit** permiten migrar y manipular datos fácilmente desde archivos **CSV** a bases de datos no relacionales como **Firestore**, usando **Python**.
- Reconociste que **Firebase** ofrece una suite de herramientas como **Realtime Database, Firestore, Cloud Functions, Firebase ML, Hosting** y **Cloud Storage** para facilitar el desarrollo de aplicaciones.
- Examinaste que para conectar **Firestore** con **Python**, se requiere descargar las credenciales del proyecto, configurar el entorno y codificar funciones para insertar, leer, actualizar o eliminar datos.
- Revisaste que **PostgreSQL** es ideal para aplicaciones estructuradas, y se destaca por su fidelidad a los estándares **SQL** y sus funcionalidades avanzadas.
- Analizaste que las bases de datos relacionales como **PostgreSQL** organizan datos en tablas conectadas mediante claves primarias y foráneas, usando **SQL** para consultas.
- Interiorizaste que **Streamlit** permite consumir datos de **PostgreSQL** conectándose con credenciales desde **Python** y visualizando resultados directamente en la aplicación web.

Bibliografía

Ou, S. (02 de agosto de 2024). *Display Live Data Using Streamlit and a PostgreSQL Database* [Visualización de datos en directo mediante Streamlit y una base de datos PostgreSQL]. Ploomber. Recuperado el 12 de junio de 2025 de: <https://ploomber.io/blog/streamlit-postgres/>

Streamlit. (2025). *Connect Streamlit to MongoDB* [Conectar Streamlit a MongoDB]. Streamlit. Recuperado el 12 de junio de 2025 de: <https://docs.streamlit.io/develop/tutorials/databases/mongodb>

Streamlit. (2025). *Connect Streamlit to PostgreSQL* [Conectar Streamlit a PostgreSQL]. Streamlit. Recuperado el 12 de junio de 2025 de: <https://docs.streamlit.io/develop/tutorials/databases/postgresql>

Streamlit. (2025). *Use Streamlit and Firestore to create a serverless web app with persistent data, written entirely in Python* [Utiliza Streamlit y Firestore para crear una app web sin servidor con datos persistentes, escrita íntegramente en Python]. Streamlit. Recuperado el 12 de junio de 2025 de: <https://blog.streamlit.io/streamlit-firestore/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 12 de junio de 2025 y su disponibilidad depende del autor.